从一个 3 开始

Digit Studio 神经网络与手写数字识别教材

看见像素 · 手算梯度 · 搭建模型 · 回到项目

面向初学者 | 完整配套 CPU 实验 | A4 双面打印版

版本 1.0 | 核对日期 2026-10-11

基线仓库：https://github.com/3128771573/digit-studio

main · 43b82e9eedae27ed9771f40df130e0e51ceed462

通用原理、教学例子、基线事实与真实实验分别标明。

阅读导航

开始之前 3
第 1 章 写下一个 3，电脑看到了什么 4
第 2 章 不用神经网络，能认出来吗 7
第 3 章 给数值一个合适的位置 8
第 4 章 第一个会学习的分类器 10
第 5 章 从十个分数到可训练的错误 12
第 6 章 一个参数怎样挪一步 14
第 7 章 亲手走完一次反向传播 16
第 8 章 为什么需要非线性与多层 18
第 9 章 卷积：把同一把尺子滑过图片 19
第 10 章 先搭一个小 CNN，再搭第二组 22
第 11 章 让训练成为可复查的实验 23
第 12 章 优化器：如何使用梯度 26
第 13 章 BatchNorm、Dropout 与正则化 28
第 14 章 怎样知道学得好不好 29
第 15 章 把自己的笔迹送进模型 31
第 16 章 回到 Digit Studio：一条请求的旅程 33
第 17 章 使用、停止与关闭：以实际实现为准 35
第 18 章 用证据回答：为什么它仍然会错 38
附录 A 学习路线与数学符号 41
附录 B 练习答案与解题思路 43
附录 C 配套实验手册与重新生成 45
附录 D 源码与参考资料 47
附录 E 基线、验证记录与修订说明 49

开始之前

Digit Studio 神经网络与手写数字识别教材

版本 1.0 · 2026-10-11 · 面向尚未系统学习 Python、微积分和深度学习的读者

这本书的起点是纸上的一笔，终点是你能解释、训练并使用自己的模型。你不需要先学完一门高等数学课；需要用到的概念，会在第一次计算之前出现。你也不必把每行程序背下来。请把“我能预测下一步会发生什么”当作学习进展的标志。

项目基线：Digit Studio，默认分支 main；完整提交 43b82e9eedae27ed9771f40df130e0e51ceed462。核对日期 2026-10-11。仓库地址 https://github.com/3128771573/digit-studio 。该仓库为私有仓库，源码链接需要相应访问权限。原始材料是仓库中的 47 页《Digit Studio 项目说明书》。本书重组学习路线，纠正容易误导的表述，保留项目使用与排错路径。

阅读约定：标为“原理”的内容适用于所注明条件；“教学模型”是独立实验代码；“基线事实”来自上述提交；“真实实验”来自随书记录；“探索”是建议读者执行的实验，不能当成已经发生的结果。

数学分三层：A 是每位读者都要理解的计算意义；B 是核心推导，建议拿纸逐步检查；C 是进一步理解，可以先跳过。公式使用自然对数 ln；数组索引从 0 开始；数学中的单样本向量默认列向量，代码中的批次每行一个样本。两种布局会明确连接。

第 1 章 写下一个 3，电脑看到了什么

一笔和一个标签，是两种东西

在纸上写一个 3。人看到的是字符，图像文件记录的是不同位置的亮度。模型接收亮度数组，输出十个数；把最大数对应的索引叫作预测类别。这一过程没有自动获得“第三个自然数”的含义。训练标签建立了图像与符号 3 的统计联系。

想象把纸盖上一张细方格网，每格只记录一个亮度。一格就是一个像素。灰度图像通常以 0 表示黑，255 表示白。MNIST 数据恰好采用黑底亮字，因此笔画位置较亮。网页画布是白底深色笔迹，必须在输入模型前转换。这个方向差异会贯穿全书：显示给人的图片，未必就是网络直接收到的数组。

像素没有厘米单位，它是编码后的亮度值。8 位整数只能容纳 0 到 255；浮点数可以表示小数。把整数除以 255，是换一种数值尺度，不是增加照片细节。放大一张 28×28 图片也不会凭空得到真实笔画。

A：先读懂一张小图片

我们先用 3×3 的示意图，而不是全部 784 个像素。下面的数值以黑底亮字为约定，范围 0 到 1。

```
0.0  1.0  0.0
0.0  0.5  1.0
0.0  1.0  0.0
```

第一行第二列是 1.0，表示很亮；中间是 0.5，表示中等亮度。这不是一个真实 MNIST 样本，只是用来读坐标的示意。图 1 把数组位置和图像方格连接起来。不要认为每个像素只有“有笔画”和“没笔画”两种状态：抗锯齿、扫描和缩放都会产生灰色边缘。

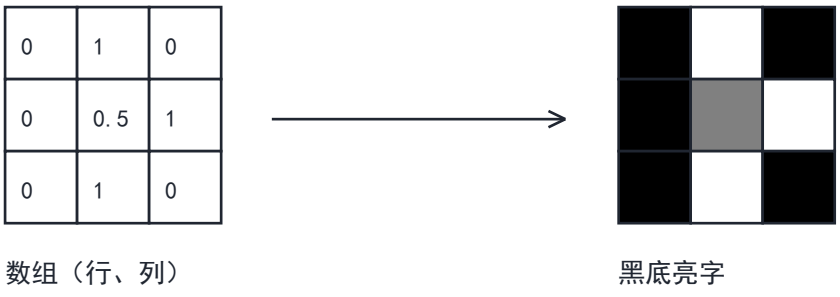


图 1 位置决定意义：同样九个数，改变排列就会改变图像。示意数据，非实验结果。

一个数叫标量；按顺序排列的一串数叫向量；有行和列的二维数组叫矩阵；推广到更多维叫张量。“张量”在本书首先表示一个带形状的数值数组，不需要先理解更抽象的几何定义。

一张 MNIST 灰度图的空间形状是 (28, 28)。PyTorch 图像加上通道维后是 (1, 28, 28)，32 张组成批次是 (32, 1, 28, 28)。这里 N 是样本数，C 是通道数，H 是高，W 是宽；通道可以是灰度的 1，

也可以是彩色图的 3。批次不是把 32 张图片拼成一幅长图。

第一段 Python：让电脑回答形状问题

先认识读代码需要的五个动作。赋值 x=2 给一个对象起名字；列表 [1,2,3] 保持元素顺序，x[0] 读取第一个元素；函数 f(x) 接收输入并返回结果；import torch 引入库，torch.tensor 表示库里的张量构造函数；for 每次处理一个元素，if 根据条件选择路径。Python 用缩进而非大括号表示属于哪段代码，通常每层四个空格。

逗号连接的返回值可以拆开：image,label=dataset[0] 表示从一对结果中分别得到图像与标签。方法是对象提供的函数，如 image.resize(...)；属性如 image.size 不带调用括号。命令行 python -m package.module 是运行一个模块，和在 Python 文件中写 import 的目的不同。

不必此刻掌握所有 Python 语法。先把“名称指向对象→函数加工对象→检查结果”串起来。以下是可以放进根目录临时 Python 文件运行的最小读取代码；它读原训练集第 0 张，和后面固定随机划分的第 0 张不一定相同：

```
import numpy as np
from torchvision.datasets import MNIST

dataset = MNIST("textbook/labs/data", train=True, download=True)
image, label = dataset[0]
pixels = np.asarray(image)
print(label, pixels.shape, pixels.dtype)
print(pixels[10:15, 10:15])
```

切片 10:15 表示索引 10 到 14，不含 15；二维逗号前是行、后是列。成功时空间 shape 是 (28,28)，dtype 是 uint8；这个原始读取没有通道维，也没有标准化。完整 inspect_data 继续做变换并检查 (1,28,28)，所以两种输出不矛盾。

工作目录始终是仓库根目录，即含 README.md 和 mnist_web 的目录；命令中的路径都相对于它。Windows 先创建 .venv，之后让 $py 指向该环境的解释器。这是一个 PowerShell 变量，不是系统设置。

```
python -m venv .venv
$py = ".venv\Scripts\python.exe"
& $py -m pip install -r mnist_web/requirements.txt
& $py -m textbook.labs.inspect_data
```

macOS/Linux 创建环境后使用 .venv/bin/python；后文命令里的“& $py”替换为这个解释器路径即可。每次打开新的 PowerShell 窗口都要重新定义 $py。若现有项目已有 .venv，不需要再创建。

第一次运行会下载 MNIST 压缩文件，约 11 MB，解压后的缓存更大。教学缓存是 textbook/labs/data，不提交到 Git。已有缓存可复制，不必再次下载：

```
& $py -m textbook.labs.inspect_data --source-cache mnist_web/data
```

此命令只读原缓存，写教学缓存和 textbook/evidence。成功后应看到 shape=[1,28,28]、dtype=torch.float32，以及真实样本图 mnist_samples.png。标准化后的最小值可以小于 0，最大值可以大于 1，原因在第 3 章。若报找不到模块，通常工作目录错了；若下载失败，先确认网络，

保留完整异常，不要随机删除权重。

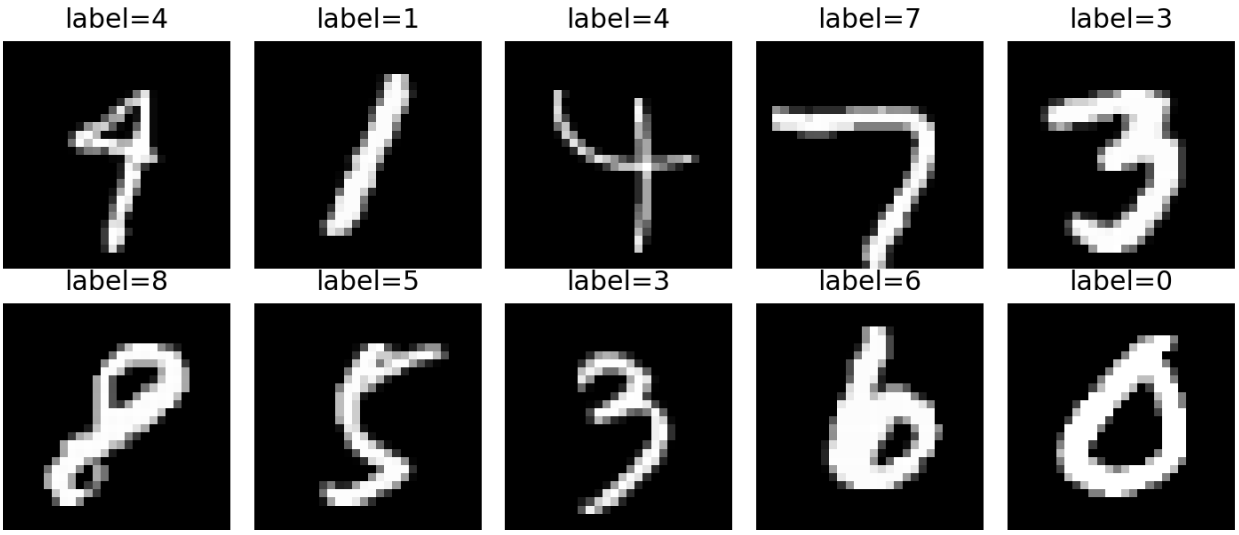


图 2 本次固定训练子集中的十幅真实 MNIST 图片，标题为数据标签；显示前反标准化，非模型预测。

这里第一次遇到 Python 的导入：import 把已经安装的功能引入程序。变量 x 是给一份数据起名字；x.shape 读取属性；print(x.shape) 调用函数。括号中是输入，结果可以赋给新变量。不要把“变量名”与“数值是否会改变”混淆。

先预测，再动手

在生成的样本图上找到一个 3。预测：它的左上角更接近黑还是白？运行 inspect_data 后看 data.json 中第一幅图左上 5×5 的值，再根据标签和位置核对。这个输出只属于固定划分下选出的样本，不是所有 3 的共同模板。

本章练习 1A：28×28 灰度图含多少个数？64 张的 NCHW 形状是什么？1B：如果把 0 和 255 对调，shape 会改变吗？1C：把一张 28×28 图片放大到 280×280，为什么不能据此声称识别信息增加了十倍？答案见附录 B。下一章我们先尝试一种不需要梯度的方法。

第 2 章 不用神经网络，能认出来吗

如果用“最像谁”来判断

如果某幅图像和已知的 3 很接近，也许它也是 3。于是可以储存十幅参考图，把新图片与每幅参考图逐像素比较。最简单的方法是平方距离：每个位置的差平方后相加。这样差的正负不会互相抵消。

$$d(x,t)^2=\sum_{i=0}^{783}(x_i-t_i)^2$$

这里 x 和 t 是 784 维向量，i 是位置索引，d² 是平方距离。距离越小，逐位置亮度越相似。对 x=[0,1]、t=[1,1]，平方距离是 (0−1)²+(1−1)²=1。对 u=[0,0]，距离也等于 1。这说明距离相同可能需要明确并列规则，代码 argmax/argmin 默认选择先出现的索引，不能把并列解释为确定证据。

只选一位同学的字体太偏。我们可以把训练集中同类的图像求平均，得到十个类别平均图。第 c 类第 i 个位置的模板是该类所有样本在 i 位置的平均亮度。

$$t_{c,i}=\frac{1}{n_c}\sum_{k:y_k=c}x_{k,i}$$

n_c 表示该类训练样本数。若该类一个样本也没有，这个式子没有定义，实验代码会报错，而不是默默给零模板。平均图会比较模糊：它汇总了各种字体和位置，不是某个真实人的笔迹。

一个成功也不能掩盖的弱点

假如把同一个 3 向右移动两格，语义标签没有改变，但对应像素发生了许多变化。平方距离看到的仍是位置差异，不知道“这是同一幅图移了一点”。先预测这个变化会怎样影响模板排序，再去实验。模板法的不足来自所选距离与表示，不是因为电脑“没有理解能力”的一句泛泛描述。

同一标签可能有多种字形；不同标签可能共享笔画。平均图把各种写法融合，可能离任何一张实际图片都不够近。规则识别也类似：人为设置“上下两个弧线”规则后，还要解决倾斜、断笔、粗细和噪声。如果规则能覆盖你的输入分布，它可以有用；并非所有任务都必须神经网络。

实验 2：建立有记录的基线

完整文件是 textbook/labs/template.py。主角是 sums，形状 (10,784)，储存各类像素总和；counts 是 (10,)，储存各类样本数；means=sums/counts[:,None]，通过增加一个维度使每类数量作用于该类所有像素。

```
means = sums / counts[:, None]
scores = -(x.flatten(1)[:, None] - means[None]).square().sum(2)
prediction = scores.argmax(1)
```

输入 x 是 (N, 1, 28, 28)，展平后 (N, 784)。差值是 (N, 10, 784)，最后沿像素维求和，得到 (N, 10)。负距离使“更大分数”对应“更近”。这些分数不是天然校准的概率；对它们强行 Softmax 可以得到和为 1 的数，却不自动给距离一个合理温度。因此配套模板结果不拿交叉熵与神经网络作公平比较。

```
& $py -m textbook.labs.template
```

成功检查点：template.json 中 counts 每项大于 0，混淆矩阵总数等于验证样本数 512。这个快速实验只使用固定训练子集 2,048 张；加 --full 可使用 55,000 张训练、5,000 张验证。看哪个类别容易混淆，但暂时不要根据测试集挑规则。

实验记录卡：问题是模板是否对平移敏感；固定训练样本、模板和距离；只改变横向位移；记录每幅图的距离排名和整体验证准确率；比较变化最大的样本。不能据此证明神经网络一定更好，也不能把一次位移实验推广到旋转、颜色或多位数字。

练习 2A：计算 [1, 2, 0] 与 [0, 2, 2] 的平方距离。2B：为什么平均图应该按类别分别求，不该先把全部标签混在一起？2C：开放探索：改用多个模板后，需要控制哪些变量才可比较？下一章先补齐数组和统计，才能把“学习参数”写成正确的计算。

第 3 章 给数值一个合适的位置

展平不会删除数值，但会改变我们如何使用位置

把 28×28 展成 784 维，像把一张棋盘按行读成一串棋格。只要记住顺序和原形状，可以 reshape 回去。展平本身不会损失像素；问题在于后续模型是否利用二维邻接关系。

```
flat = images.flatten(1)       # (N, 784), preserve N
restored = flat.reshape(-1, 1, 28, 28)
```

−1 表示让程序根据元素总数推算那一维。它不是任意值：不能把 784 个数 reshape 成 (30, 30)。flatten(0) 会把批次也混进去，破坏“一行一个样本”的约定，是常见错误。transpose 调换维度顺序；reshape 重解释元素布局，两者用途不同。

A：点积是“同位置相乘，再相加”

向量 x=[1, 2, 3] 与 w=[2, 0, −1] 的点积是 2+0−3=−1。元素乘法结果是 [2, 0, −3]，仍然是向量。Python/NumPy/PyTorch 中 * 通常表示元素乘法，@ 表示矩阵乘法。两者不能互换。

$$w^T x = \sum_{i=0}^{d-1} w_i x_i$$

T 表示转置，不是平方。w 与 x 是 d×1 列向量；w^T 是 1×d，乘 x 得 1×1 标量。矩阵乘法 (a, b)@(b, c) 得 (a, c)，中间维度必须一致。这种形状检查是低成本的排错工具。

图 3 给出代码常用的批量布局：X 是 (N,d)，W 是 (K,d)，所以 $Z=XW^{T}+b$ 是 (N,K)。b 是 (K,)，广播到 N 行。若误写 X@W，往往维度错误；某些方阵下却能运行，计算仍然错，因此“能跑”不等于“含义对”。



图 3 矩阵乘法检查。每个输出分数，来自一行样本与一行类别权重的点积。

A：均值、方差与标准差

均值把一组数的总和除以数量。方差衡量偏离均值的平方距离平均；标准差是方差的平方根，因此回到原数值尺度。对 [0,1,2]，均值 1，总体方差 2/3，标准差约 0.816497。若估计总体方差时使用无偏样本估计，分母是 n−1，本例得到 1。必须说明自己采用哪个定义。

$$\mu=\frac{1}{n}\sum_i a_i,\quad \sigma^2=\frac{1}{n}\sum_i (a_i-\mu)^2$$

本书普通标准化写成 (a − μ)/σ。项目使用固定 μ=0.1307、σ=0.3081，先把像素除以 255，再标准化。于是黑底 0 变成约 −0.424213，亮笔画 1 变成约 2.821904。这不是每幅图各自重新计算均值和方差，也不保证每张图片恰好均值 0、方差 1。

“归一化”是宽泛术语，常指把值缩放到某个范围；“标准化”在这里特指减均值、除标准差。预处理必须和训练约定匹配。如果训练看标准化数值，预测却只做 /255，网络会看到不同分布。

为什么最后一个小批次不能和大批次等权

一个批次 64 个样本平均损失 1，最后一个批次只有 6 个样本平均损失 3。总平均应为 (64×1+6×3)/70=1.171429，而不是 (1+3)/2=2。每个样本等权，才能与逐样本求和一致。

$$\bar{L}=\frac{\sum_b n_b \bar{L}_b}{\sum_b n_b}$$

这里 b 是批次索引，不是神经元偏置；上下文会区别。训练代码累加 loss.item()*len(y)，再除以 total。Accuracy 也应累加正确数量再除样本数，而不是简单平均批次百分数。

实验 3：先检查输入约定

运行教材 checks 会核对加权平均、模型形状和数值梯度。你也可以在 Python 中用三个小张量试乘法。列表 [1,2] 与 [3,4] 拼接是另一种操作；想做数值运算，应转成 torch.tensor 或 numpy.array。函数以 def 定义；缩进说明哪些语句属于函数或循环；for x,y in loader 每次取一个批次；冒号和四格缩进是语法的一部分。

```
& $py -m textbook.labs.checks
```

成功检查点：6 项机制与完整路径检查通过。它们验证明确性质，不能替代真实数据训练，也不证明高准确率。

练习 3A：(32,784)@(784,10) 的形状？3B：100 个样本准确率 90%，10 个样本准确率 50%，总准确率多少？3C：为什么用测试集重新估计标准化常量后，再反复调模型，会污染公平评估？我们现在有了表示与运算规则，下一章把模板换成可学习参数。

第 4 章 第一个会学习的分类器

权重不再由我们手写

一个神经元先计算 $z=w^Tx+b$。权重 w 决定各输入如何影响分数，偏置 b 决定所有输入为零时的起点。对 x=[2,1]、w=[0.5,−1]、b=0.2，z=1−1+0.2=0.2。权重为负不是“坏参数”，只是该特征增加时分数会减少。

$$z = w^T x + b$$

识别十类需要十个分数。将十组权重放成矩阵 W∈R^(10×784)，b∈R^10；单样本列向量 x∈R^784，z=Wx+b∈R^10。批次按行储存时，换成 $Z=XW^T+b$。PyTorch Linear(784,10) 的 weight 形状就是 (10,784)。

相邻两类 a、c 的分界满足 z_a=z_c，也就是 $(w_a-w_c)^Tx+(b_a-b_c)=0$。这是一个仿射超平面。三维以上无法直接画出来，但含义仍然是输入空间被线性分数比较分割。这里“线性分类器”沿用常见叫法，严格说含偏置的计算是仿射变换。

第一份模型只要两层容器

文件 textbook/labs/models.py 的 linear 分支是完整定义；Flatten 没有可训练参数，Linear 有 784×10 个权重和 10 个偏置，总共 7,850 个参数。

```
from torch import nn

model = nn.Sequential(
    nn.Flatten(1),
    nn.Linear(784, 10),
)
```

```
logits = model(images)    # (N, 10)
```

nn.Module 是可组合的网络组件；Sequential 按写出的顺序运行。model(images) 调用前向计算。初始权重是随机初始化的，并非模板平均图。参数与超参数不同：权重和偏置由训练更新，学习率、层宽和 batch size 则是你先选的设置。

这里 images 指从 DataLoader 得到的批次，不是 PIL 图片；Linear 的两个位置参数是输入长度、输出长度。后面 padding=1、lr=0.001 这种写法是关键字参数，用名字说明设置。完整脚本的 def make_model(name) 把模型构造放进可重复调用的函数；return 返回构造出的模型。你暂时不必自己写 nn.Module 子类，Sequential 已能完成前三种基础结构。

为什么最大分数还不够训练

argmax 能告诉我们哪一类排第一，却不能平滑描述“离正确答案近了多少”。目标类从第二名变成只差一点的第二名时，预测标签不变；训练仍应知道取得了进展。我们需要把分数转换为一组可比较数值，再用一个连续损失评估错误。下一章正是解决这个问题。

先看看完整训练命令；先不要求理解尚未讲解的 backward，其机制在第 6、7 章展开。它不是给你一份大代码后让你自行猜测，而是让同一短循环不断得到解释。

```
& $py -m textbook.labs.train --model linear --run my_linear
```

实验写入 textbook/labs/runs/my_linear；默认 CPU、2 epochs、batch=64、Adam lr=0.001、无增强、固定种子 42。它不会加载、覆盖原项目权重。目录已经存在时会报错，换一个 --run 名字保留旧实验。成功检查点：每轮打印有限的 train_loss、val_loss；生成 best.pt、last.pt、config.json、history.json、split.json。

常见错误：把 y 转成浮点数造成标签类型不符；将 10 当作合法标签导致越界，因为标签是 0–9；把批次维展平导致目标数量不匹配；训练前重复给输入标准化导致尺度错。先检查 x.shape、y.shape、x.dtype、y.dtype，比盲调学习率有效。

练习 4A：Linear(3,2) 有几个参数？4B：某像素权重从 0.1 改成 0.2，像素值 0.5 时，对应分数改变量多少？4C：如果给所有类别分数加同一个常数，预测会变吗？

第 5 章 从十个分数到可训练的错误

A：为什么用指数与归一化

logits 可以是负数，也不需要和为 1。Softmax 将它们转换成正数，再除以总和。指数函数 exp(z) 始终为正且严格递增；因此分数大小顺序被保留，并放大分数差异对应的比值。

$$p_j = \frac{e^{z_j}}{\sum_{k=0}^{K-1} e^{z_k}}$$

K 是类别数，p_j 是模型分配给第 j 类的数值。对于 [2,1,0]，指数约 [7.389056,2.718282,1]，概率约 [0.665241,0.244728,0.090031]，和为 1。若把一个 logit 增加 1，它相对另一个类的概率比会乘以 e；不是概率本身简单加 1。

大数指数容易溢出。令 m=max(z)，先减去 m。每个指数都乘了 e^(−m)，分子分母共同因子抵消，所以结果不变：

$$\frac{e^{z_j - m}}{\sum_k e^{z_k - m}} = \frac{e^{-m} e^{z_j}}{e^{-m} \sum_k e^{z_k}} = p_j$$

这时最大指数是 1，数值更稳。Softmax 的 argmax 与 logits 相同，有限输入下概率排序也相同；浮点舍入和并列仍要按实现解释。

十个类里必须分完，不代表知道世界上只有十个类

给模型一张猫、一封信或者两个数字，Softmax 仍把总量分给 0–9。高概率可能只是某类比另外九类分数高很多。它没有自动新增“未知”类别。空白检测是输入规则，不是通用未知检测；完整未知拒识需要额外数据、评价方法与设计。

比例是两个数量的比；百分数只是比例乘 100 后的表达。经验频率来自实际计数；模型概率来自当前参数下的输出。后者是否接近同类预测的真实正确频率，需要校准评估，不能因总和为 1 就宣布可信。

A：交叉熵为什么惩罚自信的错误

若真实类别是 y，希望 p_y 大。取负对数得到 L=−ln(p_y)。p_y 趋近 1 时损失趋近 0；p_y 很小时损失很大。对数把概率乘积变成和，这是从独立样本似然到训练总损失的桥梁。独立建模是一个建模假设，不是所有现实数据天然独立。

$$L = -\ln p_y = -z_y + \ln\left(\sum_k e^{z_k}\right)$$

p_y=0.9 时 L≈0.105361；0.5 时≈0.693147；0.1 时≈2.302585。每个值都必须用自然对数。如果把 ln 换成 log10，损失和梯度尺度会改变。损失不是错误百分比，不能把 0.2 loss 说成 20% 错误。

对于 N 个样本，无类别权重、无忽略标签的普通平均交叉熵为 L_batch=(1/N)∑L_n。PyTorch CrossEntropyLoss 接收原始 logits 与 int64 类别索引，内部稳定计算 LogSoftmax 和目标类别负对数。先 Softmax 再把概率交给它，会把概率当作新 logits，改变目标。

```
logits = model(x)
loss = torch.nn.functional.cross_entropy(logits, y)
probabilities = logits.softmax(dim=1)  # for inspection, not CE input
```

参考 [R1]：PyTorch 2.14 CrossEntropyLoss。带 class weight、ignore_index、软标签时平均规则不能机械套用 N；本书基础实验没有这些选项。

B：完整推导输出梯度

用上面 L=−z_y+ln S，S=∑exp(z_k)。第一项对 z_j 的导数：j=y 时 −1，其他时候 0，写成 −1[j=y]。第二项按链式法则：ln S 对 S 的导数 1/S，S 对 z_j 的导数 exp(z_j)，乘积就是 p_j。因此：

$$\frac{\partial L}{\partial z_j} = p_j - \mathbf{1}[j = y]$$

真实类的梯度通常为负：梯度下降会增大其 logit；其他类为正：下降会减小其 logit。但这些 logits 同时依赖许多共享参数，实际更新对不同样本可能产生竞争，不应把这一局部结论说成每一步所有样本都改善。

另一种检查从 Softmax 导数开始：∂p_i/∂z_j=p_i(1[i=j]−p_j)，再乘 ∂L/∂p_y=−1/p_y，得到同样结果。第一种推导更短，也解释为什么稳定实现合并运算。

批次平均的 G_nj=(p_nj−1[j=y_n])/N。除 N 必须明确；如果采用求和损失，则没有这个因子。教材 checks 的 float64 自动求导与有限差分核对这个式子。

实验 5：标签没变，损失也可能变

固定真实标签为 0，分别比较 logits=[2,1,0] 与 [3,1,0] 的损失。预测都为 0，第二组目标概率更大，损失更小。再构造多数样本刚跨过分界、一个错误变得极其自信的例子：Accuracy 可能升高而平均 Loss 升高。记录逐样本损失，才知道原因。

练习 5A：真实类概率 0.25 时损失是多少？5B：[2,1,0]、真实类 1 的单样本输出梯度是多少？5C：为什么最大概率 0.99 不是“未来 100 次必对 99 次”的保证？下一章我们把输出错误连接到参数。

第 6 章 一个参数怎样挪一步

A：导数是一把局部尺子

把参数 w 稍微改动 ε，观察损失变化。差商 [L(w+ε)−L(w)]/ε 是局部变化率的近似；导数是 ε 趋近 0 时的极限。这里不需要先做长篇极限证明，要先理解它回答的问题：当前位置向哪边小步移动，损失更可能下降？

对 L(w)=(2w−3)²/2，w=0.5 时预测 1，损失 2；导数 L′(w)=2(2w−3)=−4。η=0.1 时 w_new=w−ηL′=0.9，新预测 1.8，损失 0.72。注意这次只更新 w；若同时更新偏置，结果不同，下一节会明确。

$$\theta_{new} = \theta - \eta \nabla_\theta L$$

θ 是所有参数构成的向量，η 是学习率，梯度每个分量是损失对那个参数的偏导。梯度指向局部最陡上升方向，负梯度指向下降方向；这是在通常欧氏距离度量下的描述。一步太大可能越过低谷，不能把“方向对”说成“任意步长都对”。

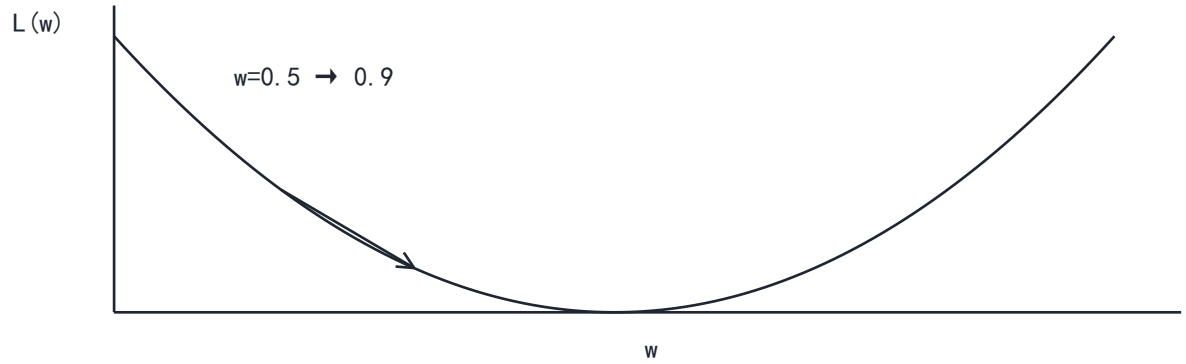


图 4 局部斜率与更新。曲线为 L(w)=(2w−3)²/2 的解析示意，箭头表示一次小步更新。

B：链式法则把分数和参数连起来

设 z=wx+b，L=(z−y)²/2。先求 ∂L/∂z=z−y，再求 ∂z/∂w=x，相乘得到 ∂L/∂w=(z−y)x；∂L/∂b=z−y。对 x=2、y=3、w=0.5、b=0，z=1，L=2，梯度 dw=−4、db=−2。

同时更新且 η=0.1：w=0.9、b=0.2；新 z=2，新 L=0.5。所有梯度必须由同一个旧参数状态计算。如果先改 w 再用新 w 计算 db，这已经是不同算法。

乘法链条像逐段追问：损失随分数变多少？分数随参数变多少？把每段比例相乘。类比边界是：分支处还要把所有路径贡献相加，多变量不是只有一条线。

有限差分是一种检查，不是实际训练方案

中心差分 $[L(w+\varepsilon)-L(w-\varepsilon)]/(2\varepsilon)$ 通常更准确，但 ε 太小会受浮点舍入影响，太大会偏离局部。检查用 float64 和 $\varepsilon=1e-6$；普通训练常用 float32。若模型有随机 Dropout，有限差分两次用了不同掩码，差值就混入随机变化，应先固定随机性或 eval。

逐个扰动数十万参数需要大量前向计算。自动求导记录前向运算关系，按链式法则反向累积导数；它不选择正确数据、损失、标签和学习率，也不保证模型学到你想要的东西。

```
optimizer.zero_grad(set_to_none=True)
loss = criterion(model(x), y)
loss.backward()
optimizer.step()
```

zero_grad 清除上一次累积梯度；backward 计算并累加到参数的 .grad；step 读取梯度更新参数。遗漏清零会把多个批次梯度相加；有时梯度累积是有意设计，但需要正确缩放，不是偶然忘了清零。

实验 6：学习率大一点会怎样

假设“小步比大步更稳”。固定初值、数据和平方损失，只改变 η 为 0.01、0.1、1，手算五步并画损失。这个一维凸问题可给出解析结论，不应推广为深网络通用最佳学习率。观察交替震荡还是单调接近，先辨别原因，再试调度器。

练习 6A：上述含偏置例子 $\eta=1$，更新后 z 和 L 是多少？6B：梯度为零意味着全局最优吗？6C：为什么 no_grad 包住训练前向之后，再调用 backward 会失败？下一章用一个分类网络完整走过更新。

第 7 章 亲手走完一次反向传播

A：一颗隐藏神经元也能展示完整机制

输入 x=2；隐藏层 a=wx+b，初值 w=0.5、b=0，所以 a=1；ReLU(a)=max(0,a)，h=1。输出层两个分数 $z_0=u_0h+c_0$、$z_1=u_1h+c_1$，u=[1,−1]，c=[0,0]，真实类别 y=1。这是两分类教学例子，不是 MNIST 网络。

前向 z=[1,−1]；Softmax p≈[0.880797,0.119203]；L=−ln(p₁)≈2.126928。它预测 0，目标却是 1。下面每个数字都由 evidence/math.json 中 float64 自动求导复算。

B：从输出依次往前

第一步：g=∂L/∂z=[0.880797,−0.880797]。

第二步：输出权重梯度 du_j=g_j h，因此 du=g；输出偏置 dc=g。

第三步：隐藏输出 h 同时影响两个分数，所以 $dh=u_0g_0+u_1g_1=1.761594$。分支贡献必须相加，不能只沿正确类的那条边传播。

第四步：a=1>0，所以 ReLU 导数为 1，da=dh=1.761594。接着 dw=da×x=3.523188，db=da=1.761594。

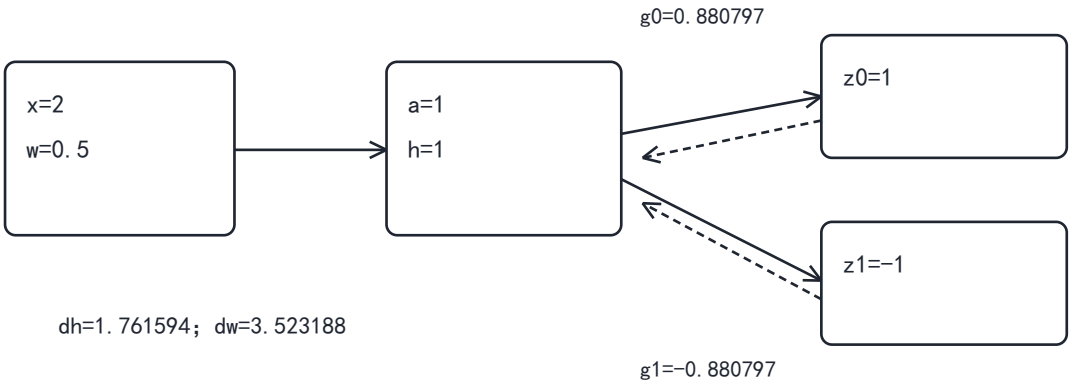


图 5 两条输出路径的梯度在 h 汇合。黑色箭头是前向值，虚线箭头是反向导数；数值为手算教学示例。

η=0.1，同步更新得到 w≈0.147681、b≈−0.176159、u≈[0.911920,−0.911920]、c≈[−0.088080,0.088080]。新 a≈0.119203，仍为正；新 logits≈[0.020623,−0.020623]，新损失约 0.71398，精确值见数值表。这是一次明确更新带来的下降，不是“以后每次都下降”的证明。

B：全连接层的通用形状

令 h∈R^d、W∈R^(K×d)、b∈R^K，z=Wh+b；g=∂L/∂z∈R^K。每个 W_ji 只直接影响 z_j，且导数是 h_i，所以：

$$\frac{\partial L}{\partial W} = gh^T, \quad \frac{\partial L}{\partial b} = g, \quad \frac{\partial L}{\partial h} = W^T g$$

g hᵀ 是 (K,1)@(1,d)，结果 (K,d)；Wᵀg 是 (d,K)@(K,1)，结果 (d,1)。形状和参数一一对应。批次按行布局时 H∈R^(N×d)、G∈R^(N×K)，dW=GᵀH、db=∑_n G_n、dH=GW。如果 G 已包含平均损失的 1/N，就不能再除一次。

ReLU 选择了哪些梯度路径

正输入导数 1，负输入导数 0，所以负区间向前一层的梯度被挡住。它不是把负参数删掉，而是本次输入经过这个单元时局部导数为零。某单元若对所有训练输入都处于负区间，可能长时间没有来自这条路径的更新。

最大池化把一个窗口的最大值传出去。反向时，输出梯度传给本次取得最大值的位置，其他位置贡献为零。若多个位置并列最大，数学上没有唯一普通导数；框架采用具体索引或子梯度规则。C 层讨论中应区分不同后端的选择，基础实验避免用并列值核验精确路径。

卷积同一权重用于多个空间位置，这些使用位置对损失的贡献在梯度中相加。一个 1D 卷积核长度 2，输入 [1,2,3]，把两个输出相加作为损失：第一个权重参与 1 和 2，梯度 3；第二个参与 2 和 3，梯度 5。共享的是参数，累计的是所有使用地点的导数，不是给每个位置新建权重。

实验 7：手算与框架互相核对

运行 checks，打开 math.json，逐项对应本章。再把目标 y 改成 0，先预测 g 的符号，再运行一个临时副本；不要修改证据文件冒充原始运行。成功检查点：手算、解析梯度和自动求导在舍入容差内一致。

练习 7A：隐藏层为负时 dw 为什么可能为零？7B：批次 N=4 时每样本输出梯度如何缩放？7C：卷积共享权重的例子若损失是两个输出平均，梯度是多少？到这里，你已经能解释基本训练循环。接下来才值得增加层数。

第 8 章 为什么需要非线性与多层

多叠几个直线变换，仍是一条直线变换

若 $h=W_1x+b_1$，$z=W_2h+b_2$，代入得到 $z=(W_2W_1)x+(W_2b_1+b_2)$。中间没有非线性时，两层可合并为一层仿射变换；堆很多层不自动扩大这种模型的函数类型。

$$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$$

经典小例子是 XOR：二维输入 $(0,0)$、$(1,1)$ 属于类 0；$(0,1)$、$(1,0)$ 属于类 1。单个仿射分界不能把对角的两点与另外两点分开。加入非线性可以组合多个区域；这说明表达能力发生了变化，不保证训练算法一定找到好参数。

ReLU 提供分段结构

ReLU 对负数输出 0，对正数保持原值。多组权重产生不同的分界，后续层再组合这些分段响应。模型可以学习输入与标签的统计模式，也可能记忆训练例子。不能用“神经网络从不记忆”来解释泛化。

现在 MLP 的增量只有一个隐藏层：784→64→ReLU→10。参数量 $(784\times64+64)+(64\times10+10)=50,890$。它比线性模型更多，但没有卷积的局部连接和共享结构。

```
model = nn.Sequential(
    nn.Flatten(1), nn.Linear(784, 64),
    nn.ReLU(), nn.Linear(64, 10),
)
```

每层输入输出依次是 $(N,784)$、$(N,64)$、$(N,64)$、$(N,10)$。ReLU 没有参数；隐藏宽度 64 是教学选择，不是理论最佳值。

```
& $py -m textbook.labs.train --model mlp --run my_mlp
& $py -m textbook.labs.evaluate --run my_mlp
```

evaluate 默认评估验证集，不碰测试集。成功检查点：best.pt 可加载，输出形状正确，验证矩阵总和 512。观察 linear 与 mlp 时固定数据划分、种子、优化器、学习率、batch、epoch 和预处理，记录耗时与参数量。相同 epoch 不代表相同计算成本，学习率相同也不代表各架构都已被公平调到最佳。

实验 8：把问题变成可检验假设

假设“非线性使这个任务在相同设置下更容易拟合”。建立带 ReLU 的 MLP，再建立两个 Linear 连续而无 ReLU 的对照；只改变非线性。记录训练与验证损失。如果结果不同，还要考虑参数化导致的优化差异，不能单凭一次结果声称已经证明完整表达能力定理。

常见排错：网络过宽而样本少，训练准确率高、验证不高；学习率过大出现发散；输入错标准化导致激活尺度异常。优先检查一小批数据能否被拟合，再逐步扩大；小批过拟合是诊断工具，不是最终成绩。

练习 8A：推导三个仿射层的合并形式。8B：为什么删掉 ReLU 后“参数仍很多”不能反驳仿射合并？8C：开放题：隐藏宽度 16、64、128 怎么比较，哪些结果不能归因于宽度本身？下一章回到图片的二维位置，看看卷积能节省什么。

第 9 章 卷积：把同一把尺子滑过图片

A：局部乘积求和

MLP 第一层给每个像素位置不同的连接权重。卷积先查看小窗口，用同一组权重在各位置重复计算。这种结构假定某种局部模式在不同位置都值得检测，因此为图像提供有用的归纳偏置。它不是提前指定“第 7 个通道必定寻找左弧线”。通道响应由训练结果决定，往往比一个命名笔画更复杂。

取输入 3×3、核 2×2：

```
input            kernel
1  2  0          1  0
0  1  3          0 -1
2  0  1
```

无填充、步长 1，输出左上 1×1+2×0+0×0+1×(−1)=0；右上 2−3=−1；左下 0−0=0；右下 1−1=0。因此输出 [[0,−1],[0,0]]。这里的核是人为设置的教学例子，训练中的核一般随机初始化并被梯度更新。

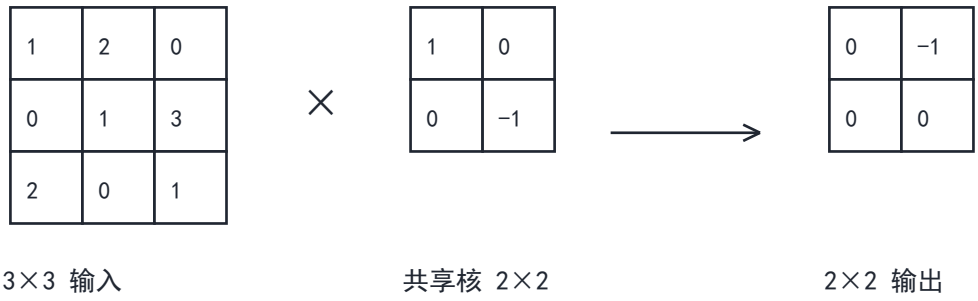


图 6 同一个 2×2 核滑过四个窗口。局部位置变了，参数没有变。示意计算。

PyTorch Conv2d 实际计算互相关，没有先把核旋转 180°；深度学习语境中仍称卷积。只要从头学习权重，两种约定可对应，但手算与程序核的方向必须一致。参考 [R2]。

B：多通道不是把结果随便叠起来

一个输出通道的核覆盖所有输入通道。若输入 C_in=3，核形状是 (3,k_h,k_w)，先分别逐位置乘积，再把输入通道与核位置的贡献求和，最后加一个输出偏置。输出 C_out 个通道就有 C_out 组这样的核。

$$Y_{o,r,s} = b_o + \sum_{c=0}^{C_{in}-1} \sum_{i,j} K_{o,c,i,j} X_{c,r+i,s+j}$$

这个式子为 stride=1、无 padding、dilation=1 的简式。一般索引要考虑步长、填充和膨胀；不要直接拿简式去解释任意配置。groups=1、有偏置时参数数为 C_out(C_in k_h k_w+1)，与图片宽高无关。

1→8、3×3：8×(9+1)=80 个参数。8→16、3×3：16×(72+1)=1,168。一个输出通道不是只对应一个输入通道，除非专门采用分组或深度卷积；本项目没有使用这些结构。

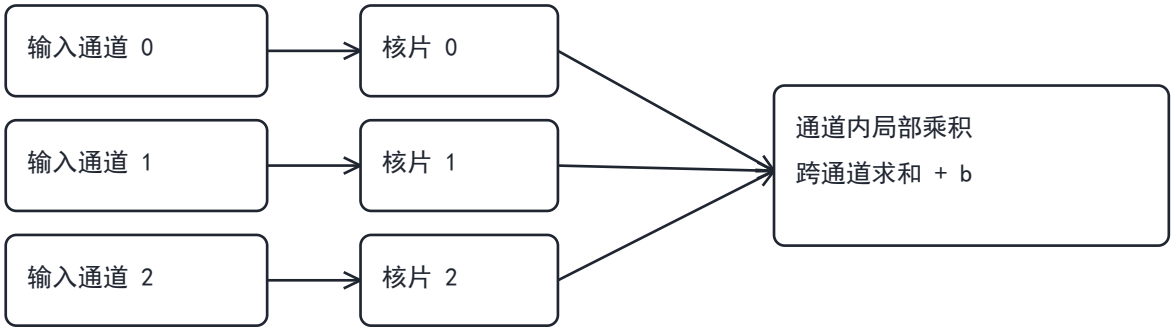


图 7 一个输出通道汇总所有输入通道的局部乘积。多个输出通道各自拥有一组核。

输出尺寸：先算，再运行

单个空间方向输入长度 H，核长 k，填充 p，步长 s，膨胀 d，则输出长度为：

$$H_{out} = \left\lfloor \frac{H + 2p - d(k-1) - 1}{s} + 1 \right\rfloor$$

padding 在边缘补值，常用零；stride 决定窗口移动多少格；dilation 在核元素间留间隔，其有效长度为 d(k−1)+1。对 H=28,k=3,p=1,s=1,d=1，输出 28。对 p=0，输出 26。对 28,k=3,p=1,s=2，输出 14。二维高宽分别计算。

边缘窗口含补零而非真实图像，因此卷积的平移等变性通常在无限网格或适当边界条件下成立；有限图片、padding、stride、后续池化和展平分类层都会影响系统的平移响应。等变指输入平移时特征跟着平移，不等于最终类别完全不变。

池化和感受野

2×2 最大池化取窗口最大值。例如 [[1,4],[2,3]] 输出 4；它减少空间分辨率，没有可训练参数，也会丢失位置和非最大值信息。不能把“降低尺寸”说成没有损失信息。

定义 r 为某层一个位置覆盖的输入理论感受野大小，j 为相邻位置在原输入上的间隔。初始 r=1, j=1；有效核 k_eff=d(k−1)+1 时，r_new=r+(k_eff−1)j，j_new=js。padding 改变位置偏移和边界有效范围，不改变内部位置这个理论递推。

项目两组 3×3 卷积、2×2 池化：r 从 1→3→4→8→10，j 从 1→1→2→2→4。第二次池化一个位置理论上覆盖 10×10 输入；后续全连接组合所有位置，最终类别依赖更大范围。有效感受野的实际贡献通常并不均匀，理论覆盖不等于每个像素贡献相同。

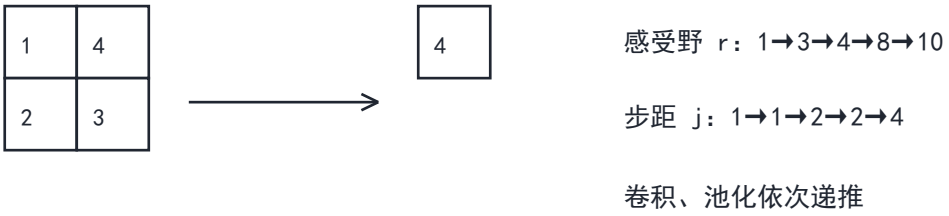


图 8 最大池化与感受野递推。最大值保留、其他值丢失；反向梯度沿本次最大值位置传递。

实验 9：先把核当作测量工具

固定上述输入和核，手算再用 Conv2d 核对；改变 padding 后观察边缘输出。再将核权重设为全 1，输出变成局部和。实验只能解释这些指定核的计算，不能证明训练后每个通道的含义。

练习 9A：32→64、3×3 有偏置卷积有多少参数？9B：输入 14，核 3，padding 1，stride 1 后再池化 2，输出尺寸？9C：卷积之后右移的特征图为何仍不保证分类器预测不变？下一章把一组卷积接成可以训练的模型。

第 10 章 先搭一个小 CNN，再搭第二组

第一次只增加必要结构

教学 cnn1：Conv2d(1, 8, 3, padding=1) → ReLU → MaxPool2d(2) → Flatten → Linear(1568, 10)。没有 BatchNorm、Dropout、调度器，便于观察结构带来的改变。它的形状如下：

位置	输出形状	可训练参数
输入	(N, 1, 28, 28)	0
卷积	(N, 8, 28, 28)	80
ReLU、池化	(N, 8, 14, 14)	0
展平	(N, 1568)	0
分类	(N, 10)	15, 690
总数		15, 770

```
model = nn.Sequential(
    nn.Conv2d(1, 8, 3, padding=1), nn.ReLU(),
    nn.MaxPool2d(2), nn.Flatten(1),
    nn.Linear(8 * 14 * 14, 10),
)
```

```
& $py -m textbook.labs.train --model cnn1 --run my_cnn1
& $py -m textbook.labs.evaluate --run my_cnn1
```

文件仍是 models.py、train.py、evaluate.py，训练循环保持一致。成功检查点：每个模型返回 (N, 10)，参数量核对通过，验证报告生成。常见错误是改了池化或 padding，却仍保留旧 Linear 输入长度；先用 dummy=torch.zeros(2, 1, 28, 28) 跑一次。

第二组为什么可能减少总参数

教学 cnn2 在第一个池化后增加 Conv2d(8, 16, 3, padding=1)、ReLU、第二次池化。展平长度从 1568 变成 16×7×7=784，最终分类层从 15, 690 减为 7, 850；总参数 80+1, 168+7, 850=9, 098。层数更多不代表参数必然更多。

```
nn.Conv2d(8, 16, 3, padding=1), nn.ReLU(),
nn.MaxPool2d(2), nn.Flatten(1), nn.Linear(16 * 7 * 7, 10)
```

这段是替换 cnn1 的尾部，不是重复增加两次 Flatten。完整正确版本在 models.py。输入依次 (N, 8, 14, 14) → (N, 16, 14, 14) → (N, 16, 7, 7) → (N, 784) → (N, 10)。

```
& $py -m textbook.labs.train --model cnn2 --run my_cnn2
& $py -m textbook.labs.evaluate --run my_cnn2
```

增加第二组扩大局部特征组合范围，同时压缩位置。对于小数字图可能有用，但某些细微区别也可能被池化损失。不要把“深一些”当成免检理由。

实验 10：把位置变化变成曲线

固定训练好的模型、验证数据和权重，只改变水平平移 dx=−4, −2, 0, 2, 4；用黑底对应的标准化值填空白，不使用 torch.roll 的环绕效果。运行：

```
& $py -m textbook.labs.shift --run my_cnn2
```

输出 shift.json，每个位移的样本数都是 512。分别对线性、MLP、CNN 做同样实验。观察验证准确率随平移变化，以及哪些数字发生切边。因为改变的输入既含位移也可能含裁切，这不是纯粹证明数学平移等变性的实验；它评估的是具体有限图像系统。

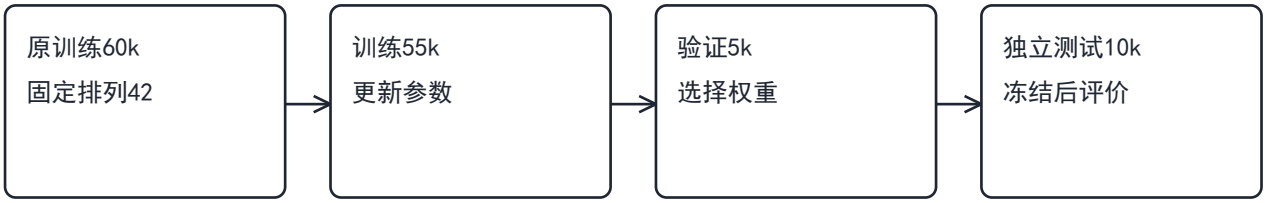
练习 10A：cnn2 参数为何少于 cnn1？10B：让第二层 padding=0 后应怎样重算分类层长度？10C：开放探索：加数据平移增强能否改善曲线？只改增强，重新训练同种架构，重复多个种子后才讨论稳定性。

第 11 章 让训练成为可复查的实验

训练、验证、测试，分别回答不同问题

训练集用于更新参数；验证集用于选超参数与选择权重；测试集用于在选择结束后估计表现。它们不能混在一起。反复观察测试结果再挑网络，相当于让测试信息参与选择，最终数字不再是一次独立检验。

项目基线从 MNIST 原训练 60,000 张中，用固定种子 42 的排列选前 5,000 张为验证，后 55,000 为训练，原测试 10,000 张保留。教学实验沿用这个划分方法；快速模式只取训练划分前 2,048、验证前 512，以及测试前 1,000。快速测试前缀不是类别均衡抽样，不能和项目的每类 100 张混淆矩阵直接当同一成绩。



分工与顺序要明确，不能把中间步骤隐藏为“自动识别”。

图 9 数据流分工。训练更新参数；验证选择配置与 checkpoint；测试在选择冻结后使用。

划分依据索引，不只是三个数量。split.json 记录真实索引，config.json 记录哈希、种子、架构、设备、版本、标准化、优化器、耗时和参数量。记录固定种子有助于复现，不保证跨设备、跨版本逐位相同；并行计算和数值舍入都会造成差异。参考 [R3]。

每轮循环其实做了五件事

1. model.train() 选择训练行为；2. 从 loader 取批次；3. 前向计算 logits 与损失；4. 清梯度、反向、更新；5. 在独立验证数据上 eval 并记录。样本顺序用私有 torch.Generator 固定，DataLoader num_workers=0 便于 Windows 与 CPU 基础实验。

```
model.train()
for x, y in train_batches:
    optimizer.zero_grad(set_to_none=True)
    logits = model(x)
    loss = F.cross_entropy(logits, y)
    loss.backward()
    optimizer.step()
```

此段要先 import torch.nn.functional as F。完整 train.py 使用全名 torch.nn.functional，所以不依赖这段别名。train_loss 是训练过程中不同参数状态下各批次损失的加权平均；val_loss 是轮末固定参数在验证集的评估。两者不是完全相同的测量条件。

epoch、batch 和更新次数

epoch 是遍历训练数据一轮；batch 是一次输入的样本集合；每个批次通常对应一次优化器更新。2,048 张、batch=64，每轮 32 步；batch=128，每轮 16 步。相同 epoch 下，更大 batch 会减少更新次数；比较 batch 时应说明是否固定样本遍历次数、更新次数或时间预算。

项目 train_one_epoch 正确按样本数量汇总损失与正确数，但停止时有零批次边界，不能把 n=0 的占位 loss=0/acc=0 视为实际学习成绩。第 17 章会讨论这个基线细节。

基线联系：[S3] get_dataloaders 确认划分与增强；[S5] train_one_epoch/evaluate 确认加权统计与模式；教学模型始终使用独立 train.py。

保存、加载与断点续训是不同目标

教学包分别保存 best.pt 与 last.pt。best 使用验证准确率严格改善时的深复制权重，记录对应 epoch；last 是最后一轮参数。保存 config 包含模型名字、输入形状、标准化与图片处理版本。load_checkpoint 先构建同架构，再 load_state_dict，最后 eval。

```
checkpoint = torch.load(path, map_location='cpu', weights_only=True)
model = make_model(checkpoint['config']['model'])
model.load_state_dict(checkpoint['state_dict'])
model.eval()
```

只加载可信文件。weights_only=True 限制常见反序列化对象范围，也不应该被描述成对任意文件绝对安全。原项目明确使用 weights_only=False，因此不要从陌生来源下载 pth 再直接加载。参考 [R4]。

这些 checkpoint 足以推理，但不支持严格恢复下一训练步：还缺 optimizer/scheduler 状态、随机源状态、采样器位置与更多环境记录。严格续训尤其要区分中途批次、完整轮末、最佳验证权重；不能只凭“保存了权重”声称完整训练现场保留。

原项目保存/加载见 [S7] save_model/load_model；教学包与原项目权重格式、架构和加载限制有明确区别，不应直接互换。

实验 11：冻结选择再测试

把验证选择标准写在运行前：在同一快速设置的四个模型中选验证 Accuracy 最高者；并列时沿固定模型顺序取第一个。只对它运行 --split test。记录“2 epochs 快速检查，单次种子”，不可包装成 MNIST 最终性能结论。

```
& $py -m textbook.labs.evaluate --run my_cnn2 --split test
```

这是示范命令，只有你在验证选择后确定使用 cnn2 时才执行。成功检查点：结果中 split=test、n=1000（快速）或 10000（full）；checkpoint_epoch 明确。

练习 11A：55,000 张、batch 64、drop_last=False，每轮多少批？11B：为何最佳验证指标需要与具体权重对应？11C：删除优化器状态再继续训练 Adam，会丢失什么？

第 12 章 优化器：如何使用梯度

A：梯度下降和小批次梯度

普通梯度下降用整个训练集的平均梯度更新；随机或小批次梯度下降用部分样本估计。小批次噪声可能帮助也可能妨碍优化，不能把它说成必然有益。数据顺序、batch size 和学习率一起决定路径。

动量保存过去梯度的指数衰减累计，使方向不必完全跟随本次噪声。教学简式 v_t=βv_(t−1)+g_t，θ_t=θ_(t−1)−ηv_t。β=0 回到普通更新。也有人在第一式乘 (1−β)，那改变了统计尺度；公式约定需说清。PyTorch SGD 有初始化、dampening、Nesterov 等细节，本项目设 momentum=0.9，未启用 Nesterov。

B：Adam 的两份记账

Adam 跟踪梯度的一阶指数平均 m 和平方梯度的二阶指数平均 v。平方不是方差本身；不减去均值平方时，v 是未中心化二阶统计。初始 $m_0=v_0=0$，早期平均被零初值压低，用偏差修正：

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$

以上为逐元素运算。$\beta_1=0.9$、$\beta_2=0.999$，首次 g=2：$m_1=0.2$、$v_1=0.004$；修正后 $\hat{m}_1=2$、$\hat{v}_1=4$；若忽略很小 ε，更新量约 η。负梯度会向相反方向更新。ε 防止极小分母，并影响数值，不能在代码中随意删除。

Adam 的自适应尺度不是自动寻找最佳学习率，仍需要外部 η。不同论文和实现可能把 ε 放在不同位置、采用不同融合路径。参考原始论文 [R5]，本书基本公式不是完整 API 算法规格。

RMSprop 与 weight decay

RMSprop 主要跟踪平方梯度的衰减平均，用它缩放当前梯度；可选动量和中心化形式。本项目使用 PyTorch 默认非中心化 RMSprop。不要把“只跟踪第二份统计”说成与 Adam 其余行为完全相同。

平方参数惩罚 $\lambda||\theta||^2/2$ 的梯度是 λθ。把它加入普通 SGD 梯度，更新包含 $(1-\eta\lambda)\theta$，因此与简单乘法衰减相联系。对于 Adam，若先把 λθ 加进 g，再进入 m、v，自适应缩放会作用于惩罚项；AdamW 将衰减与梯度统计分开，效果一般不同。

$$\theta_{new} = (1 - \eta\lambda)\theta - \eta\,\mathrm{AdamGradientStep}$$

该式中的 AdamGradientStep 表示偏差修正后的梯度比例，不应再乘一次 η。基线 build_optimizer 使用 Adam(weight_decay=1e−4)，未设置解耦衰减；它不是 AdamW。教学实验 Adam(weight_decay=0) 是为了最初隔离机制。学习时别把这两个设置拼成一个不存在的模型。参考 [R6]、[R7]。

调度器不是准确率开关

基线默认 cosine，在 epochs>1 时，轮末 scheduler.step；最低学习率 1e−5。StepLR 的步长 max(1,epochs//3)，gamma=0.3。history 的 lr 是 step 后值，更接近下一轮学习率，不一定是这一轮实际使用的值。选择 none 或 epochs=1 没有调度器。

这些项目设置来自 [S4] build_optimizer/build_scheduler 和 [S6] fit，普通机制与框架默认值分别说明。

实验 12：只改变学习率或优化器中的一个。若比较 Adam 与 SGD，同时把 SGD lr 变大，就无法把全部差异归因于优化器；可以为每个优化器采用同样预算的验证搜索，但必须说明搜索过程。记录 loss、accuracy、时间与失败运行，而不是只保留最好的一张曲线。

练习 12A：首次 Adam 梯度 −3 的修正 m、v 是多少？12B：为什么二阶统计不是标准差？12C：如果初始 η 小于 cosine 的 eta_min，是否还能把整个过程笼统称为单调降低？

第 13 章 BatchNorm、Dropout 与正则化

学会训练后再添加这些组件

正则化泛指限制拟合方式、改变目标或增加约束，使训练结果更适合某种泛化需求。它可能牺牲训练拟合，也可能改善验证表现；数据、模型和强度不同，结论不同。更深、更多参数、更大 batch、更多 epoch、更多正则化都不保证更好。

B：BatchNorm2d 到底在哪些维度统计

输入 (N,C,H,W) 时，每个通道在 N、H、W 上汇总统计。对通道 c，M=NHW，先求 μ_c 和方差 σ²_c，然后 x̂=(x−μ_c)/sqrt(σ²_c+ε)，再输出 γ_c x̂+β_c。γ、β 每通道两个可学习参数；ε 避免分母问题。

$$\hat{x}_{n,c,h,w} = \frac{x_{n,c,h,w} - \mu_c}{\sqrt{\sigma_c^2 + \varepsilon}}, \quad y = \gamma_c \hat{x} + \beta_c$$

PyTorch 训练前向使用有偏方差估计；更新 running_var 使用无偏估计。默认 momentum=0.1 表示 running_new=0.9×running_old+0.1×batch_stat，不是优化器动量同一个公式含义。推理默认用运行统计；训练用当前批统计并更新运行量。参考 [R8]。只有批大小 1 仍可能有 H×W 多个数，但极小有效样本统计可能不稳。

项目两层 BN 通道为 32、64，γ β 参数分别 64、128；running mean/var 是缓冲区，不计入可训练参数数。它们仍然必须保存到 state_dict。教科书中的简化公式应包含这些输入与模式条件。

B：Dropout 保持的是期望，不是每次结果

项目 Dropout(p=0.25)：丢弃概率 p，保留概率 q=0.75。训练时独立掩码 m∈{0,1}，保留概率 q，输出 mh/q；评估时直接输出 h。E[mh/q]=E[m]h/q=qh/q=h。保持的是固定 h 条件下的期望，不是每次样本数值都相等。

$$\tilde{h} = \frac{mh}{q}, \quad E[\tilde{h} \mid h] = h$$

若 h=6，q=0.75，保留时输出 8、丢弃时输出 0，平均期望为 6。它不是永久删除神经元，也不是训练后要把权重再乘一次 0.75；这是 inverted dropout 的约定。参考 [R9]。

eval 和关闭梯度，各自解决一个问题

model.eval() 切换 BatchNorm、Dropout 等模块行为；torch.no_grad()/inference_mode() 控制自动求导与推理开销。eval 不会自动禁止梯度；no_grad 不会自动让 Dropout 停止随机丢弃。因此推理一般同时 eval 和关闭梯度。模型没有这些模块时 train/eval 可能数值一样，但不能据此

认为两者可以互换。

实验 13：复制一个带 BN 与 Dropout 的小网络，同一输入分别 train、eval 前向，记录输出差异与 running_mean。固定权重和输入，只改变模式；不要在原项目全局模型上做 train 模式实验，避免改其运行统计。不能据此说明随机输出是错误，也不能比较不同单次掩码来宣布精度。

练习 13A：C=16 的 BN 有多少可训练参数？13B：q=0.8、h=4，保留时输出与期望？13C：在 no_grad 内运行 train 模式 BN，会不会更新 running_mean？

第 14 章 怎样知道学得好不好

A：先看计数，再看百分数

Accuracy=正确样本数/总样本数。它回答当前抽样分布下总体正确比例。若某类特别多，总体成绩可能掩盖少数类失败。混淆矩阵 M_ij 表示真实 i、预测 j 的样本数；行是真实，列是预测。对角线正确，非对角线帮助找具体混淆。

$$\text{Accuracy} = \frac{\sum_c M_{cc}}{\sum_{i,j} M_{ij}}$$

教学矩阵 [[8,2,0],[1,6,3],[0,1,9]] 有 30 个样本、23 个正确，Accuracy=76.6667%。类 2 的真实数量 10，预测为 2 的数量 12，TP=9，FP=3，FN=1。

$$P_c = \frac{TP_c}{TP_c + FP_c}, \quad R_c = \frac{TP_c}{TP_c + FN_c}$$

$$F1_c = \frac{2P_cR_c}{P_c + R_c}$$

类 2 Precision=9/12=0.75，Recall=9/10=0.9，F1=0.818182。Precision 问“预测成它的有多少真是它”；Recall 问“真是它的有多少被找出”。不同任务对两类错误的成本不同。

	预测：0	1	2	
真实 0	8	2	0	总数 30
真实 1	1	6	3	正确 23
真实 2	0	1	9	Accuracy=23/30

图 10 三类教学混淆矩阵。用数字和行列标签表达含义，不依赖颜色。此图不是 MNIST 实测成绩。

零分母不能略过

某类从未被预测，Precision 分母为 0；某类根本不在样本里，Recall 分母为 0。这些指标数学上未定义。报告可以采用 zero_division=0，但必须标注政策并另列 support；也可显示“未定义”。F1 可直接写 2TP/(2TP+FP+FN)，分母为 0 时同样需要政策。

宏平均对各类指标等权；加权平均按 support 权重；总体 Accuracy 对每个样本等权。均衡抽样与自然分布下的总体 Accuracy 含义不同。项目每类 100 张探索矩阵共 1,000 张；训练后报告是完整测试 10,000 张；顶部指标则是历史验证记录，三者不可混称一个准确率。

少量样本有多不确定

10 个独立代表性样本对 8 个，80% 只是这次比例。即使采用二项抽样假设，Wilson 95% 区间也约为 49.0% – 94.3%，很宽。网页随手挑的 10 个样本还可能不满足随机代表性，区间并不能修复抽样偏差。

校准需要检查输出概率和真实频率是否匹配，例如把最高概率分箱，对每箱同时记录平均概率与正确比例。分箱少样本同样不稳；MNIST 上的校准不自动适用于你拍摄的手写照片。参考校准论文[R10]，基础包没有实现温度缩放，不把它写成当前功能。

曲线与错误样本，回答不同问题

训练 Loss 下降而验证 Loss 上升，提示过拟合或训练验证分布差异，但应先检查指标计算、模式和预处理。Accuracy 近乎不变而 Loss 下降，可能是排名未变但目标概率增加。Loss 上升而 Accuracy 升高，也可能是少数高置信错误抵消大量小改善。

错误图集中看几类原因：笔迹歧义、图像切边、粗细差异、处理反色、样本标签与可见图形不一致。特征图提供观察内部响应的窗口；不能把一张亮区域当成“该通道对最终 3 贡献的严格证明”，尤其各通道单独归一化后亮度尺度已不同。

实验 14：在验证错误图中先提出假设，再查看 processed input。固定模型，对同一原图只改变一种预处理选项。记录预测排序、概率和像素；不能凭挑选成功样本推出总体改进。开放题评估依据是实验控制、记录完整性与结论边界。

练习 14A：上述类 0 的 Precision、Recall？14B：一个类别 support=0 时，如何写报告才不误导？14C：为什么训练集 Accuracy=100% 不能证明泛化？

第 15 章 把自己的笔迹送进模型

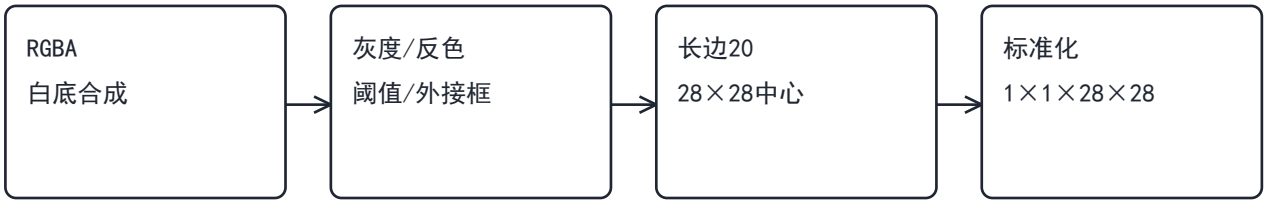
用户图片为什么更容易出错

MNIST 是特定采集与处理条件下的单字符图像。鼠标粗笔、倾斜拍照、纸张阴影、边框、两个数字、背景杂物可能形成分布变化。预测自己的图片前，应先问“模型实际收到的 28×28 是什么”，而不是直接增加 epoch。

教学脚本要求一张白底深色的单数字图片；合成示例 synthetic_three.png 是程序画的折线，不是用户真实笔迹，也不代表完整用户图片评测。

A：处理流水线分开做，才看得见问题

1. 解码为 RGBA，透明部分与白底合成；2. 转灰度；3. 反色，得到黑底亮字；4. 除以 255；5. 把 ≤0.12 的弱亮度置零，其余保留原值；6. 取非零外接框；7. 等比缩放，长边 20；8. 放入 28×28 黑底中心；9. 固定标准化；10. 增加批次、通道维。



分工与顺序要明确，不能把中间步骤隐藏为“自动识别”。

图 11 从原图到张量。显示预览是 0–1 灰度，网络输入是标准化后的 (1, 1, 28, 28)，两者不要混用。

```
display, tensor = prepare(image_path)
model.eval()
with torch.inference_mode():
    probabilities = model(tensor).softmax(1)[0]
prediction = int(probabilities.argmax())
```

成功检查点：processed_input.png 中只有完整单数字、黑底亮字、合理留白；prediction.json 有十项概率，和约为 1，形状 [1, 1, 28, 28]。模型错认示例并不表示脚本不能运行，应区分路径验证和识别质量。

```
$img = "textbook/labs/examples/synthetic_three.png"
& $py -m textbook.labs.predict --run my_cnn2 --image $img
```

教学白底模式没有自动背景判断；黑底图应先按约定转换，不要额外反色两次。空白在阈值后报错，不会送一个零输入后假装识别。应用的空白规则另见下一章。

外接框中心与亮度质心不同

外接框只看有笔画的最小矩形；几何居中把矩形放在画布中心。亮度质心为 $\bar{x}=\sum x I(x,y)/\sum I(x,y)$、$\bar{y}=\sum y I(x,y)/\sum I(x,y)$，较亮或较粗区域权重更大。字形偏重一侧时，两种中心不同。

基线 `_center_by_mass` 名字和注释说质心，但实现按裁剪后宽高摆放，未计算亮度加权质心。基线 `top=14−new_h//2`；教学 `top=(28−new_h)//2`。当 `new_h` 为奇数时，两者可能相差 1 像素。因此“流程相近”不等于逐像素一致，教材不把独立包误称项目预处理的完全复制。

实验 15：分离预处理错误与模型错误

问题：笔迹识别失败是否由尺度引起？固定原图、模型权重、反色与标准化，只改变 `auto_center` 或目标框大小。保存每个处理预览，记录预测。如果某个选项改善这一幅，仍不能说明普遍更好；建立独立的用户笔迹验证集，保留标签，才可评估范围。

常见错误：PNG 透明背景变黑；浮点 0－1 当整数 0－255；把彩色 `shape(H,W,3)` 直接喂 NCHW；忘记 `eval`；使用旧架构加载新权重；先标准化再裁剪阈值导致背景判断异常。诊断顺序：文件→灰度→反色→裁剪→缩放→标准化→shape→模型→概率。

练习 15A：为什么阈值处理不是把所有笔画像素置 1？15B：按质心居中就能保证平移不变吗？15C：自己的十张数字能否用来不断改参数后又报告为独立测试？

第 16 章 回到 Digit Studio：一条请求的旅程

我们现在能够读懂哪些新增设计

教学 cnn2 有 8、16 通道和一个分类层；基线 MnistCNN 有 32、64 通道，两层 BN，隐藏全连接 128，Dropout=0.25，再到 10 logits。不是把所有高级组件一口气塞进第一次 CNN，而是在已经理解基本路径后逐项对照。

基线位置	形状	参数量
conv1，1→32	(N, 32, 28, 28)	320
bn1	(N, 32, 28, 28)	64
第一次 pool	(N, 32, 14, 14)	0
conv2，32→64	(N, 64, 14, 14)	18,496
bn2、第二次 pool	(N, 64, 7, 7)	128
flatten	(N, 3136)	0
fc1，3136→128	(N, 128)	401,536
Dropout	(N, 128)	0
fc2，128→10	(N, 10)	1,290
合计		421,834

约 95.2% 的参数在 fc1。这提示“大模型”的主要来源不一定是卷积。BN 缓冲统计不计入上述可训练参数。原注释中的“通常 99%+”不是本次重新训练结论；模型元数据 best_acc=0.9922、epoch=5 是历史记录，不能保证对应最终权重同样表现。

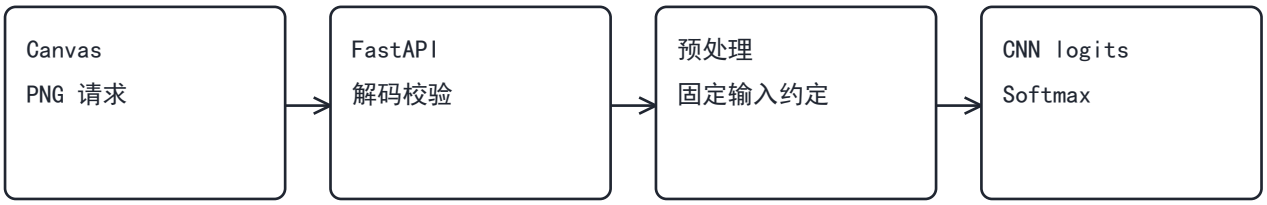
源码证据 [S1]：mnist_web/model.py，MnistCNN.__init__/forward；count_parameters。固定提交链接见附录 D，每个链接同时给相对路径与符号名。

从网页落笔到十类概率

static/index.html 提供结构，style.css 提供视觉，app.js 控制 Canvas、请求与图表，不需要 Node 构建。画布真实分辨率 320×320，坐标按显示尺寸换算；笔划结束后 debounce，延迟约 140 ms 发请求。version 用于忽略已经过时的响应，不等于服务器取消了计算。

数据经 PNG Data URL 进入 /api/predict。app.decode_image 检查前缀、Base64 解码及像素数上限，透明图与白底合成后转灰度。请求字符串最大 6,000,000 字符、图片最多 4,000,000 像素；这是基本输入校验，不是已完成全面安全审计或绝对“安全解码”。

请求行为对应 [S8] decode_image/infer/sample/analyze；页面行为对应 [S11] app.js 与 [S12] index.html，外观规则见 [S13] style.css。



分工与顺序要明确，不能把中间步骤隐藏为“自动识别”。

图 12 浏览器、FastAPI、预处理、模型的职责。预测只读当前加载模型；训练后台更新独立新模型，完成时再重载。

infer 若灰度动态范围 ptp<12.75，返回 blank=true、prediction=null、十个 0。这里概率并没有和为 1，因为系统明确表示未执行分类；不能拿这组值当 Softmax 输出。非空进入 preprocess_uploaded_image：边框平均亮度 >0.5 才反色（启用自动判断时），边框角点重复计入平均；深色边框或笔迹碰边可能误判。

auto_center=true 走阈值、外接框、长边 20、几何摆放；false 直接把整图缩至 28×28。两个分支都继续标准化。preprocess_uploaded_image(invert=False) 实际禁止自动反色，不是强制“白底按正确方式处理”。其返回第二项实际是 torch.Tensor，旧类型注释不准确。源码证据 [S2]。

MODEL_LOCK 保护推理与模型替换，inference_mode 关闭求导；predict_batch 又设置 eval 并计算 softmax。接口 elapsed_ms 从解码之后开始，包含预处理、锁等待和推理，不包含整个 HTTP 往返或浏览器绘图。界面显示的耗时不可与端到端延迟混称。

探索页看到的是什么

特征图来自 relu(bn1(conv1(tensor)))，是第一层 BN、ReLU 后且池化前，形状 (32,28,28)。前 8/16/32 通道各自按 min/max 归一化，再变 PNG；亮度不能直接跨通道比较。探索矩阵取测试集每类最早 100 张，均衡、确定性共 1,000，不是随机抽样全部测试。

样本快捷按钮从测试集抽图，反标准化再反色给画布显示，画布缩放后重新进入上传预处理，所以和直接在官方测试张量上评估不是完全相同链路。差异应分析像素处理，不应伪造两种路径准确率相等。

多个网页共享什么

每个标签页有自己的画布、撤销和页面状态；后台 MODEL、TRAINING、STOP 是同一进程共享的。一个页签启动训练，另一个页签会看到同一任务；一个关闭服务，所有页签失去后端。刷新页面丢失当前画布不等于停止任务。该锁设计以单进程服务为前提，不应未经改造直接套到多 worker 或多用户服务器。

实验 16：开启两个页签，只读状态并比较；在一个页签改画布，另一个不会同步笔迹。若要测试训练共享，请先备份权重与报告，因为原项目训练会覆盖它们。不能把本章理解任务扩展成修改应用或安全审计。

练习 16A：为什么第一层特征图没有缩到 14×14？16B：origin 校验能证明用户身份吗？16C：为何样本按钮返回正确标签，不代表应用已经把标签传给分类器？

第 17 章 使用、停止与关闭：以实际实现为准

安装与启动

回到仓库根目录，用第 1 章同一 .venv 安装依赖。Windows 双击 mnist_web/start.bat，脚本先切换到自身目录，优先找根目录 .venv；服务 started 后自动打开 http://127.0.0.1:7860。直接命令：

```
$py = ".venv\Scripts\python.exe"
& $py mnist_web/app.py --open-browser
```

浏览器打开是后台检查服务器 started 后的操作，不是网页自己启动 Python。关闭浏览器不会关服务；已运行时再打开地址即可。服务默认绑定 127.0.0.1、端口 7860，项目是本机应用。命令 python -m uvicorn app:app 绕开 main 的 server 注册，网页关闭会返回 503，应终端 Ctrl+C。

识别、训练、探索的完整使用路径

识别：写单个数字，查看答案、次选概率与模型输入预览；错误先看预览。训练：先备份 mnist_cnn.pth 与 outputs，设 epochs/batch/lr/optimizer/scheduler，再启动。它默认新建 MnistCNN，不从当前权重续训；网页训练会覆盖固定权重文件。探索：选择图片，看第一层特征与可选均衡矩阵；特征图是观察，不是严格归因。

每轮结束才向网页 yield 曲线更新；源文件开头遗留“每若干 batch 推送”“Plotly”等注释不对应现实现。训练批进度回调主要给终端；前端轮询训练状态约 1.5 s，不意味着曲线每 1.5 s 都有新批次结果。网页预测模型是在 done 更新后重载，不是每个磁盘 checkpoint 保存后立即替换。

启动依据 [S15] start.bat、[S10] main；训练生成器见 [S6] fit，重载与停止见 [S9] training_worker/stop_training。

保存成功要独立核查

train.fit 每轮和最后保存当前模型，best_acc 只是历史最大验证 Accuracy；没有单独保存那次最佳权重。save_model 直接 torch.save 到目标文件，没有临时文件原子替换。保存异常可能只记录警告，任务继续结束；因此“任务结束”“running=false”“停止已收到”“服务已关闭”都不能充当保存成功证明。

实际核查：看日志是否保存成功；比较文件时间与大小；在可信环境重新加载，核对元数据与一次前向；重要实验保留旧备份和哈希。时间变了仍不是文件完整性证明，重载成功也不等于恢复了最佳模型或完整优化器现场。

停止发生的阶段决定结果

下面基线边界来自 fit、train_one_epoch、training_worker 与独立受控检查。不要把“停止并保存”按钮文案解释成所有分支相同。

停止时点	基线行为	不能承诺
数据准备阶段	准备仍可能完成，轮前检查 STOP；无 history 时保留旧权重	立即停止下载或立即关闭
进入轮次之前	轮前 STOP 检查跳过训练，无 history 保留旧权重	已有新训练模型
轮前检查之后、首批检查之前	可出现 n=0；仍验证、追加 history 并保存新初始化模型	零批次必然保留旧权重
部分批次完成后	批边界退出，仍验证、记录部分轮次、保存当前参数	完整 epoch 或最佳权重
验证阶段	evaluate 不检查 STOP，等待验证结束	立即取消验证
报告阶段	报告路径不检查 STOP，可能完成全测试及绘图	报告立即中止或全部成功
保存异常	记录警告后可能继续 done、释放任务锁	done 必然意味着持久化成功

零批次边界很少见，但源码并未把 n>0 作为追加 history 的条件。独立检查用临时模型和小数据模拟这个时间窗口，确认不会碰原权重。该结论不是修复建议已经被实现；原项目本次未改动。

关闭和重启

网页右上角“关闭服务”会请求 /api/shutdown，置 CLOSING、STOP，阻止新训练；后台等待 JOB_LOCK 释放，再 server.should_exit=true。它等的是训练线程结束，包括可能的报告与保存失败处理，不是直接验证“保存已成功”。Origin 若存在必须等于本服务地址；缺少 Origin 的请求

仍可通过，这个检查不是身份认证。

前端收到 closing 后停止常规轮询并禁用界面，再探测状态。请求失败可能来自网络错误，不能独立证明进程终止。更可靠的关闭确认是启动终端打印“服务已关闭”，进程退出，端口不再被该服务监听。终端 Ctrl+C 也可关闭；强制结束进程可能中断写入，不用于日常停止。

重启重新运行 start.bat；启动加载磁盘文件，不恢复上次画布或任务。若端口占用，先查看旧终端和监听进程，避免随机杀所有 Python。PowerShell 可检查：

```
Get-NetTCPConnection -LocalPort 7860 -State Listen
```

备份建议：关闭训练后把 mnist_web/mnist_cnn.pth、outputs、requirements.txt 与提交 SHA 放到你自选的备份目录，记录用途。教材不使用某台电脑的绝对路径作为安装说明。教学 runs 与应用 outputs 分开备份，避免把 quick 权重替换为生产权重。

排错从最短路径开始

找不到 torch：确认实际解释器属于 .venv；缺库：对该解释器 pip install；端口冲突：核对旧进程；下载失败：核对缓存和网络；页面有框但没结果：查 /api/status 是否 ready，读后台异常；所有字都同一类：看反色、标准化和加载架构；保存失败：查写权限、磁盘、路径与日志，保留旧文件。

模型坏了不要先删掉唯一权重。复制备份，再用独立路径训练或检查。直接删权重会失去可用模型；重新训练并不一定重现历史指标。图像不可信或权重来源不可信时，不要混淆基本图片校验和任意反序列化安全性。

练习 17A：为什么关闭页签不停止训练？17B：若 best_acc 上升后下降，最后 pth 对应哪一轮？17C：报告阶段按停止为何仍会等？17D：如何证明新模型已经加载，而不仅是文件出现？

第 18 章 用证据回答：为什么它仍然会错

识别能力从哪里来

输入像素携带字形的空间变化，参数化模型把变化映射成类别分数，损失让标注样本的目标类更有优势，梯度把输出误差连接到每个参与计算的参数，优化器重复更新。卷积让局部计算在不同位置共享，层间非线性让局部响应可以组合。这些机制共同构成识别能力，不是某一层突然获得数字意识。

泛化依赖训练分布、样本覆盖、归纳偏置与优化结果。模型可能同时学到有用规律和偶然关联。对 MNIST 有效，不保证对纸上拍照、特别粗的笔迹或陌生字形有效；十分类系统仍会对许多未知输入给出高概率。

本次真实快速实验

下表由 comparison.json 和实际日志自动填入，不是历史 PDF 成绩。训练固定 seed=42、CPU 4 线程、Adam lr=0.001、batch=64、2 epochs、无数据增强；训练 2,048、验证 512。训练速度是本次设备观测，不是读者电脑的耗时承诺。

教学模型	参数量	最佳验证 Accuracy	验证 Loss	训练秒数
linear	7,850	86.3281%	0.535284	1.212
mlp	50,890	89.4531%	0.435175	0.559
cnn1	15,770	88.0859%	0.438363	0.665
cnn2	9,098	86.5234%	0.440383	0.744

模板验证 Accuracy=82.4219%；距离分数没有作为公平交叉熵比较。

表中验证最佳权重由同一选择规则得到。不同架构参数与计算成本不同；这只是可运行路径与初步对照。完整基线 55k/5k/10k、多个种子、预算一致的超参数选择，本次没有执行，不能宣称最终 MNIST 准确率。

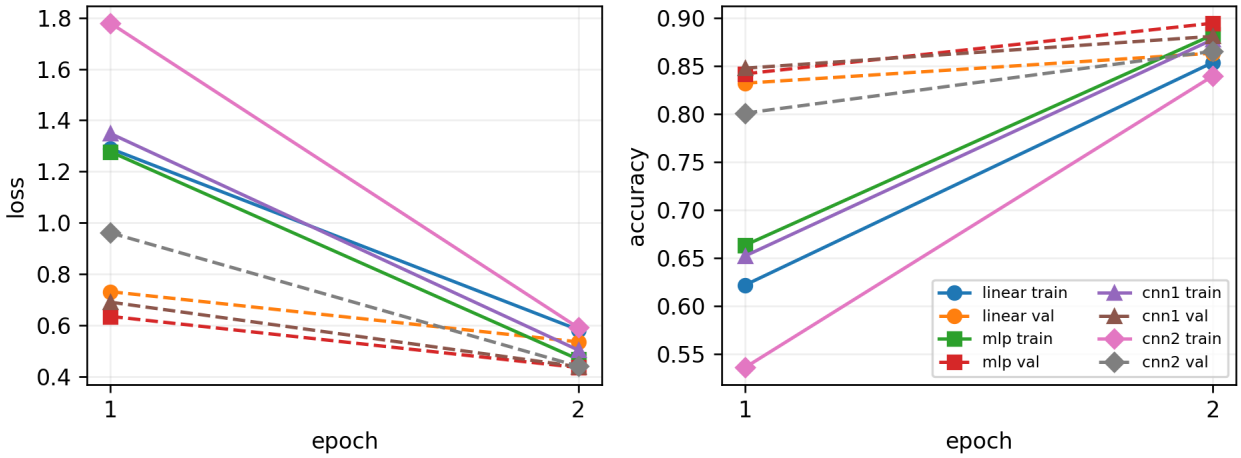


图 13 本次四种教学模型的真实训练/验证曲线。实线和虚线区分测量阶段，点形区分模型；黑白打印可读。

按验证选中 `mlp`；快速测试 n=1000，正确比例 85.2000%，平均交叉熵 0.485252。仅此一组测试评价，不是生产权重或个人笔迹准确率。

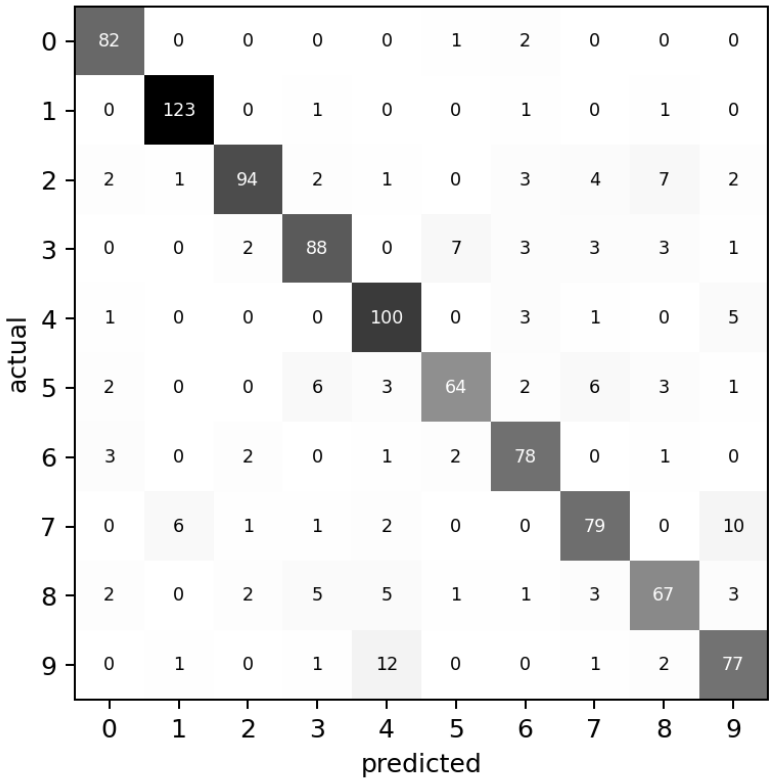


图 14 验证选择冻结后，所选模型在快速测试子集上的真实混淆矩阵。行真实、列预测，与项目探索均衡矩阵抽样不同。

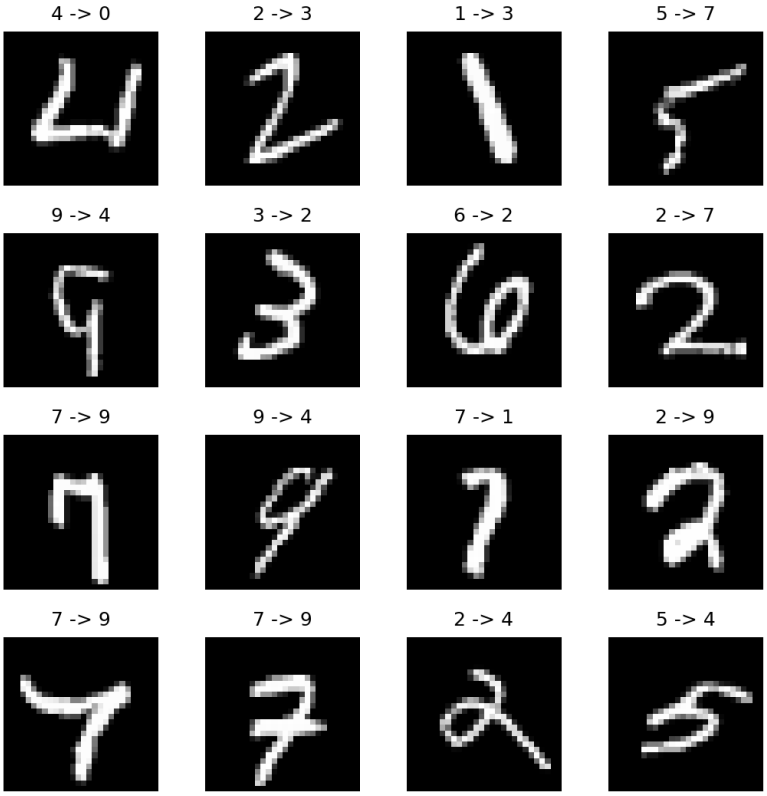


图 15 同一快速测试中的前 16 个错误样本。标题“真实→预测”，选择的是错误，不代表整体频率。

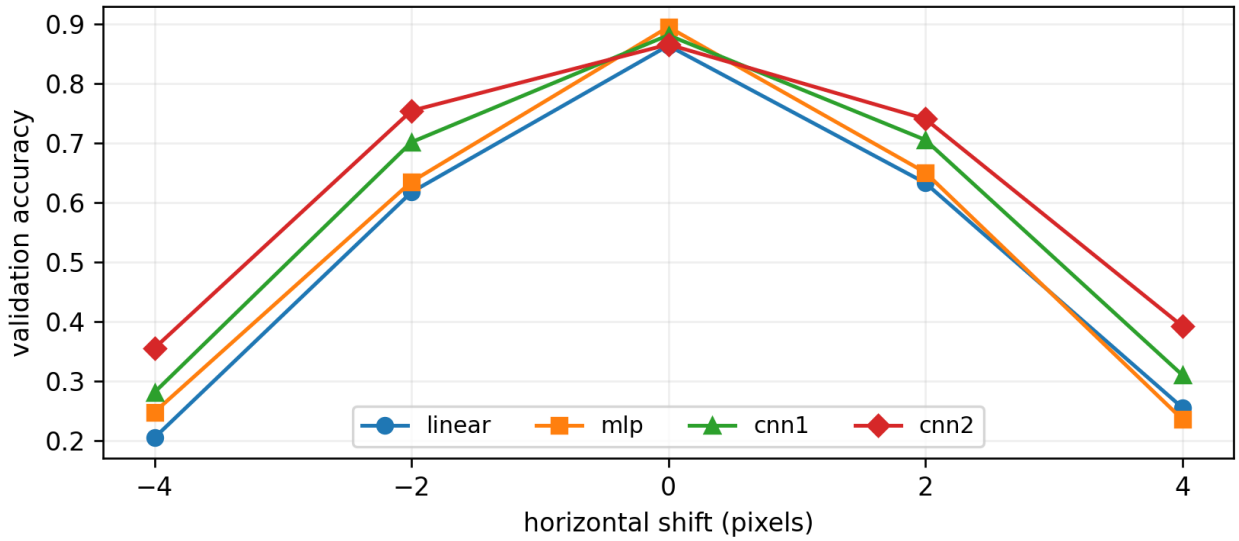


图 16 固定教学模型的真实验证平移实验。填充值对应黑底，位移可能造成裁切；此图不证明系统平移不变。

合成 3 的预测只用于文件加载、处理、保存预览、推理路径检查，不作为用户手写准确率。`evidence/prediction.json` 保留真实输出；`processed_input.png` 展示模型实际输入。失败示例也是有用证据，不能删掉不符合预期的结果。

把排错改成实验记录

一份好记录包含问题、可推翻的假设、固定条件、改变变量、操作、全部结果、可能解释、结论限制。例如“粗笔迹失败是否由笔画占比过大”要固定模型与原图，只改一次笔宽，并记录每次处理

预览；不要同时换模型、换阈值、换缩放，然后归功于任意一个因素。

实验提议 A：标签打乱。固定图片和结构，打乱训练标签，观察训练与验证。若网络拟合随机标签，是记忆能力证据；不意味着正常训练必然没有规律。实验提议 B：少样本过拟合。固定 32 张，训练较久；用于诊断循环和表达能力，不作为最终泛化结果。实验提议 C：未知输入。输入空白、符号、多数字，记录置信度，观察闭集限制；不要用最大概率阈值一次成功宣布解决未知识别。

这些提议尚未运行；评价依据是控制、可复核代码、记录与边界，而不是预设必须得到某条曲线。

最后一项综合任务

独立选择一个教学架构，写下验证选择规则，建立至少一组对照，训练并保存；冻结选择后测试；再收集自己的单数字图片，给出标签、预处理预览、预测与失败分析。最后找到 Digit Studio 对应函数，说清两套模型、数据增强、保存策略与预处理的异同。

完成标准：能解释张量形状与参数量；能推导基本梯度；能正确区分模式与求导；实验目录独立；权重可加载；结果真实；未知输入边界清楚；项目事实有固定源码链接。准确率高不是唯一评分条件，低分但诚实且诊断清楚的实验同样有学习价值。

附录 A 学习路线与数学符号

一张路线卡

路线一：第 1–3 章，读图与数值，完成 inspect_data、template、checks。你应能画出 NCHW，并区分 * 与 @。路线二：第 4–8 章，学习线性、损失、梯度、MLP，手算一次完整更新。路线三：第 9–13 章，卷积、训练规范、优化器与模式，完成 cnn1/cnn2。路线四：第 14–18 章，评价、自己的图片、项目连接与综合任务。

每次学习可以按“先预测一项结果→运行→解释差异→记下新问题”循环。时间由基础与设备决定，没有强制一周完成承诺。核心推导可用纸笔慢算；C 层内容不阻挡主线。

符号	含义与常用形状	注意
N	批次样本数量	不一定等于完整数据集大小
C,H,W	通道、高、宽	W 在网络式中也可指权重矩阵
d,K	特征长度、类别数	MNIST d=784,K=10
x,X	单样本列向量(d,1)、批量行布局(N,d)	布局不同须转置
w,W	单神经元权重(d,1)、多类权重(K,d)	T 表转置

符号	含义与常用形状	注意
b	偏置(K,)或批次索引	上下文区分
z,Z	logits(K,1)、批量(N,K)	不是概率
p,y	类别概率向量、真实类别索引	y∈{0,⋯,K−1}
L,g,G	损失、单输出梯度、批输出梯度	平均缩放不能重复
θ，η	参数集合、学习率	θ 学习，η 外部选择
μ，σ²，σ	均值、方差、标准差	说明总体/样本估计
ε，λ	稳定常数、惩罚或衰减系数	不同式子作用不同
β₁，β₂,m,v	Adam 衰减率、一阶/二阶统计	β 不同于 BN 偏移量
r,j	理论感受野、原输入步距	不是实际归因强度
1[条件]	条件真时 1，否则 0	不是文字标签

平方写 a^2，转置写 a^T；两者图形和含义都不同。正文显示公式由数学渲染器生成，不依赖中文字体提供所有数学符号。求和 Σ 的下标表明累积范围，ln 表示自然对数。

C：校验时常用的三条小推导

ReLU 分段复合产生分段仿射函数，但 Softmax 与交叉熵不再是仿射，分类边界与损失形状要分别讨论。最大池化同样是分段选择，非光滑点应避开精确有限差分检验。

卷积参数共享的梯度：若同一参数 θ 在多个运算节点 a_q 中使用，$\partial L/\partial\theta=\sum_q(\partial L/\partial a_q)(\partial a_q/\partial\theta)$。这就是第 7 章 [3,5] 的一般形式，二维、多通道只增加求和索引，不改变链式法则。

Softmax 共同平移不变带来一个检查：单样本交叉熵输出梯度的和为 $\sum p_j-\sum 1[j=y]=1-1=0$。代码若偏离太多，应检查 batch 平均、标签或是否传错概率。

附录 B 练习答案与解题思路

第 1–3 章：表示与基线

1A：784；(64, 1, 28, 28)。1B：形状不变，亮暗约定变了。1C：插值增加采样格数，无法增加原图未记录的真实细节；对特定显示或模型输入适配有用，但不是信息量必增的证明。

2A：$1^2+0^2+(-2)^2=5$。2B：十类混合平均无法保留类间差异，通常只剩一个模糊参考。2C：应固定训练与评估索引、预处理、距离、模板数量搜索预算；多个模板有更高存储与计算成本，按时间或资源一并记录。实验结果不是唯一答案。

3A：(32, 10)。3B：95/110=86.3636%，不是 70%。3C：从测试集估计输入分布后再根据测试成绩调整，选择过程已使用测试信息；应提前用训练统计定义预处理，验证选择，测试保留给最终检验。

第 4–5 章：分数与损失

4A：$3\times2+2=8$。4B：$0.1\times0.5=0.05$，其他参数固定。4C：类别排序不变，Softmax 也不变。

5A：$-\ln(0.25)=1.386294$。5B：[0.665241, −0.755272, 0.090031]，总和约 0；批次平均还要除 N。5C：模型输出不是已验证校准频率；样本分布和独立性也可能不同，有限样本仍有随机性。

第 6–7 章：梯度与一次更新

6A：w=4.5, b=2，z=11，L=32，远高于初始 2；大步越过低谷。6B：可能局部极小、极大、鞍点、平坦区或 ReLU 无梯度区，不保证全局最优。6C：no_grad 下通常没有构建求导图，loss 不需要梯度；应让训练前向保持求导。

7A：ReLU 在负区间导数 0，这条路径的链式乘积为 0；若另有路径或正则化项，参数仍可能得到其他梯度。7B：每样本 $(p-\text{one_hot})/4$。7C：[1.5, 2.5]。完整数值：新 logits=[0.020623855684, −0.020623855684]，新 L=0.713983692881。

第 8–10 章：非线性与卷积

8A：$W_3W_2W_1x+W_3W_2b_1+W_3b_2+b_3$。8B：多参数可改变优化参数化，但函数族仍是仿射；表达能力与优化便利性不同。8C：同划分、多种子、固定预算，说明相同 epoch 与时间差异；别把一条最佳运行代表总体稳定性。

9A：$64\times(32\times3\times3+1)=18,496$。9B：14 再到 7。9C：边界、采样、池化、展平分类都会破坏严格性质；局部响应平移不保证整体预测恒定。

10A：第二次池化减少分类层输入，省下参数超过新增卷积。10B：14→12→6，$16\times6\times6=576$，Linear 输入 576；总参数 $80+1,168+(576\times10+10)=7,018$。10C：比较增强前后同架构、多种子、

同预算，检查处理后图像、曲线与验证错误；提高或下降都是合法观察。

第 11 – 13 章：训练与模式

11A：ceil(55000/64)=860，最后 24 张。11B：否则读者可能把历史最大指标错误归于最后权重；实际保存策略必须单独说明。11C：Adam 的 m、v、步数及偏差修正进度，路径会改变；也还要随机源和采样状态。

12A：$\hat{m}_1=-3$、$\hat{v}_1=9$；未修正 $m_1=-0.3, v_1=0.009$。12B：v 是平方梯度平均，未中心化；标准差需方差平方根，概念和量纲不同。12C：不可以，余弦公式会从初始值趋向设定最低值，设定高于初始就不是一般所说下降；检查实际学习率序列。

13A：γ 和 β 各 16，共 32；运行统计另算缓冲区。13B：保留时 5，丢弃 0，期望 4。13C：会，no_grad 控制求导记录，不切换 BN 模式。

第 14 – 15 章：指标与图片

14A：Precision=8/9≈0.888889，Recall=8/10=0.8。14B：support=0，Recall 未定义；若政策置 0，显式说明，宏平均时说明包含/排除规则。14C：训练结果使用过这些样本更新参数，可能记忆，不代表新样本。

15A：≤0.12 置零，其余保持灰度；binary 变量名不等于全部值是 0/1。15B：不能保证，插值、裁切、采样误差和后续网络仍影响；还会改变原始信息。15C：不能，反复用于选择的集合应叫验证；另保留未参与选择的测试。

第 16 – 18 章：项目与综合任务

16A：取的是第一池化之前的响应。16B：不能，Origin 是请求来源检查，缺失也可能被允许，不是登录凭证。16C：标签用于界面与取样，预测接口收到的是 PNG 和设置；分类器不能从界面展示标签推断真值。

17A：浏览器是客户端，训练在线程与服务进程里。17B：最后保存当前最后轮次；best_acc 是历史最大，不是最佳 checkpoint 承诺。17C：报告不检查 STOP。17D：等 worker done 后重载，检查 /api/status 元数据并做可信文件前向；如需强保证应设计独立保存/加载确认，当前没有完整事务状态。

第 18 章综合任务没有唯一数字答案。建议评价：解释正确 30%、实验控制与可追溯 25%、完整可运行路径 25%、失败分析与结论边界 20%。不得因没达到某个预设准确率删改记录。

附录 C 配套实验手册与重新生成

文件怎样衔接

阶段	文件/命令模块	输入→输出	成功检查点
1	inspect_data	缓存→样本图、data.json	shape 与真实像素可读
2	template	训练子集→类别均值→验证	十类都有样本
3	models、train --model linear	(N,1,28,28)→(N,10)	7,850 参数
4	train --model mlp	784→64→10	50,890 参数
5	train --model cnn1/cnn2	局部图→10 logits	15,770/9,098 参数
6	evaluate	best.pt→矩阵和错误图	split、n、epoch 明确
7	predict	白底单数字→预览、十类概率	预处理可见、sum≈1
8	project_edges	受控基线小数据→停止证据	原权重哈希不变

common.py 提供固定划分、加载、加权评价，models.py 保持网络短小；不是要求读者先理解大框架。训练循环完整放在 train.py，首次循环在第 6、11 章逐项解释。没有隐藏 notebook 状态，所有模块从根目录运行。

快速模式用于先跑通；正式实验逐个训练可加 --full 与更充分 epochs：

```
& $py -m textbook.labs.train --model cnn2 --run full_cnn2 --full
```

默认仍 2 epochs，可单独加 --epochs 5；不要把 2 轮 full 当成充分调参。提前计划模型、种子、预算、验证规则。修改网络要新增模型名字，否则 checkpoint 中架构标签会失去含义。

实验记录表模板

问题/假设：写成可被结果反驳的句子。固定：索引、种子、设备、版本、预处理、架构、训练预算。变量：一次一个，或提前说明多因素设计。操作：完整命令和代码提交。记录：所有曲线、指标、时间、错误、失败日志。分析：结果支持什么、替代解释是什么。限制：不能证明什么，需要哪些

额外数据。

环境与运行限制

教材基础实验采用 CPU，不要求 CUDA。依赖声明沿用项目，但它是范围，不是已锁环境；实际验证版本见附录 E。data 和 runs 被 Git 忽略；evidence 是本次真实记录的轻量副本，保留日志、配置与图，不上传下载缓存。原应用的权重与 outputs 未更改。

run_verified.py 自动执行快速课程，须使用尚未存在的 verified_* 目录。如果重跑，应先把旧 runs 备份到另外名称或修改脚本的运行前缀，不强制覆盖。它在四种模型验证选择之后仅对所选模型测试一次。此脚本依赖已有 mnist_web/data 缓存复制；首次纯新环境按 inspect_data 下载，再单独运行各阶段，或去掉 --source-cache 参数。

可编辑源文件与 PDF 重建

正文是 textbook/book.md；build_book.py 读取 Markdown、插入真实结果、渲染公式与矢量机制图，生成 output/pdf/Digit_Studio_Textbook.pdf。figures.py 生成真实曲线；evidence 中 JSON 是结果来源。修改正文不需要改原项目。

PDF 构建需 reportlab、matplotlib、numpy、pillow、pypdf、pdfplumber。推荐另建 .pdfvenv，避免给训练依赖引入不必要冲突：

```
python -m venv .pdfvenv
$pdfpy = ".pdfvenv\Scripts\python.exe"
& $pdfpy -m pip install -r textbook/pdf_requirements.txt
& $pdfpy textbook/build_book.py
& $pdfpy textbook/qa_pdf.py
```

图表已提供在 evidence，重新生成无需再训练或安装 torch。中文字体默认 Windows SimHei；其他系统设置环境变量 TEXTBOOK_CJK_FONT 指向可嵌入的中文 TrueType 字体。不要填写教材作者机器路径。代码与数学使用内置/DejaVu 字体，构建器检查缺字，缺字时报错。

完整 PDF 检查还需 Poppler pdftoppm。qa_pdf.py 做页数、文本边界、链接与书签检查；视觉检查另用渲染页，不能只凭提取文本声明版式无误。双面 A4 打印按实际大小 100%，长边翻页；内侧页边距更宽，页码在外侧。屏幕 PDF 同时有目录内部链接和书签。

附录 D 源码与参考资料

固定提交源码索引

下面每条均固定完整 SHA，既给相对路径与符号，也提供永久链接；若仓库改为公开或权限变动，URL 的提交定位仍不随 main 移动。

[S1] mnist_web/model.py · MnistCNN.__init__/forward; count_parameters

[S2] mnist_web/utils.py · _center_by_mass; preprocess_digit; preprocess_uploaded_image

[S3] mnist_web/utils.py · get_transforms; get_dataloaders

[S4] mnist_web/train.py · build_optimizer; build_scheduler

[S5] mnist_web/train.py · train_one_epoch; evaluate; predict_batch

[S6] mnist_web/train.py · fit：每轮/最后保存、报告与停止

[S7] mnist_web/model.py · save_model; load_model

[S8] mnist_web/app.py · decode_image; infer; sample; analyze

[S9] mnist_web/app.py · training_worker; launch_training; stop_training

[S10] mnist_web/app.py · finish_shutdown; shutdown_service; main

[S11] mnist_web/static/app.js · 画布、请求、轮询与关闭界面

[S12] mnist_web/static/index.html · 三工作区页面与静态资源入口

[S13] mnist_web/static/style.css · 视觉样式、布局与响应式规则

[S14] mnist_web/tests/test_api.py · ApiTests：9 项回归路径

[S15] mnist_web/start.bat · 虚拟环境选择与 --open-browser

[S16] mnist_web/requirements.txt · 依赖声明范围

本书使用源码确认教学相关行为，未做整个项目安全审计。Numberly 是原稿提及的界面参考，不是本教材项目仓库；本书没有使用它的权重或把它的提交当 Digit Studio 基线。

通用原理参考资料

[R1] PyTorch 2.14，CrossEntropyLoss：logits 输入、类别目标和 reduction 规则。
https://docs.pytorch.org/docs/2.14/generated/torch.nn.CrossEntropyLoss.html

[R2] PyTorch 2.14，Conv2d：互相关、输出形状、groups 及参数约定。
https://docs.pytorch.org/docs/2.14/generated/torch.nn.Conv2d.html

[R3] PyTorch 2.14, Reproducibility：跨版本、平台复现限制。
https://docs.pytorch.org/docs/2.14/notes/randomness.html

[R4] PyTorch 2.14, Serialization semantics：state_dict 与 weights_only 限制。
https://docs.pytorch.org/docs/2.14/notes/serialization.html

[R5] Kingma 与 Ba, Adam: A Method for Stochastic Optimization, 原始论文 2014/2015。
https://arxiv.org/abs/1412.6980

[R6] Loshchilov 与 Hutter, Decoupled Weight Decay Regularization, 2017/2019。
https://arxiv.org/abs/1711.05101

[R7] PyTorch 2.14, Adam 与 AdamW 的具体实现选项。https://docs.pytorch.org/docs/2.14/generated/torch.optim.Adam.html

[R8] PyTorch 2.14, BatchNorm2d：统计维度、biased/unbiased 方差与运行统计。
https://docs.pytorch.org/docs/2.14/generated/torch.nn.BatchNorm2d.html

[R9] PyTorch 2.14, Dropout：训练缩放和 eval 行为。
https://docs.pytorch.org/docs/2.14/generated/torch.nn.Dropout.html

[R10] Guo 等, On Calibration of Modern Neural Networks, 2017。
https://proceedings.mlr.press/v70/guo17a.html

[R11] LeCun 等, Gradient-Based Learning Applied to Document Recognition, 1998；卷积学习的经典背景。https://ieeexplore.ieee.org/document/726791

[R12] torchvision 0.29, MNIST 数据集 API；数据为原训练 60k、测试 10k，划分与教学子集另由本书定义。https://docs.pytorch.org/vision/0.29/generated/torchvision.datasets.MNIST.html

参考文档版本不是依赖声明范围。原始数学推导为本书直接展开，不以长篇翻译或复制替代解释；论文用于机制出处，框架文档用于具体实现条件。

附录 E 基线、验证记录与修订说明

仓库与原材料

仓库 https://github.com/3128771573/digit-studio ，默认 main；编写开始固定 SHA 43b82e9eedae27ed9771f40df130e0e51ceed462，2026-10-11 核对远程 main 与本地一致。最终教材提交是新增交付版本，原项目事实仍引用该旧 SHA，不偷换基线。

原 PDF 是 output/pdf/Digit_Studio_Project_Manual.pdf，47 页，SHA256 b61851887aae8ce4c971c00b46cc2fe10453e925352f12a8e032500f3ac70955；文字源 output/pdf/manual_content.md。原稿已有部分谨慎说明，仍以源码消除保存、质心和测试口径歧义；不把所有原稿内容都判为错误。

声明范围、安装版本、参考文档分栏

依赖	基线声明	实际安装	文档参考
torch	>=2.4.0	2.14.0+cpu	2.14
torchvision	>=0.19.0	0.29.0+cpu	0.29
numpy	>=1.26.0	2.2.6	未用作机制版本依据
pandas	>=2.0.0	3.0.6	未用作机制版本依据
fastapi	>=0.115,<1	0.141.1	未用作机制版本依据
uvicorn	>=0.30,<1	0.54.0	未用作机制版本依据
matplotlib	>=3.8	3.10.7	未用作机制版本依据
pillow	>=10	12.0.0	未用作机制版本依据
torchinfo	>=1.8	1.8.0	未用作机制版本依据
httpx	>=0.27	0.28.1	未用作机制版本依据

验证平台 Windows、Python 3.13.7、CPU；未验证 CUDA/MPS/macOS/Linux 安装组合。requirements 并无完整锁定文件；本次安装记录在 environment.json 与 installed_requirements.txt，不能承诺任意未来解析组合兼容。PDF 构建运行时独立记录，和训练环境不混用。

本次运行与原稿记载分开

原稿记载：2026-10-11，9 项 unittest 通过，未重训现有权重。本次另行重跑这 9 项，通过；6 项教学检查通过；四模型快速训练、保存、加载、验证，验证选择后测试一次；平移、合成图片和受控停止检查完成。完整范围见 evidence/verification.md。

受控停止情形	实际处理样本 n	history 轮数	替换临时权重
before_epoch	未进入训练	0	否
zero_batch	0	1	是
partial_epoch	2	1	是
report	4	1	是

原应用权重 SHA256 前后相同：bfe1568e95a6ce5824f009ac1ea43055934e3dd246d09835edfa78340ce5899d

未跑完整 55k 训练、现有权重全 10k 重测、真人代表性评测或全屏幕浏览器验收。应用 ≥8/10 随机样本测试是路径回归，非精度承诺。TestClient 的 httpx 兼容弃用警告不影响这次通过，未来升级需关注。

应用测试定义对应 [S14] ApiTests，依赖声明对应 [S16] requirements.txt；实际安装与本次教学运行另外记录。

停止检查用 4 张数据、batch=2，打桩准备/预览/报告，真实训练、评价、保存执行；不是整库报告或端到端进程关闭。零批次 n=0 而 batches_done=1，该字段是迭代索引，不是完成更新数。

修订与原稿差异

重组：由项目文件导读改为像素→模板→线性→梯度→MLP→CNN→评价→应用。补充：完整交叉熵梯度、一次小网络反向更新、形状与参数量、优化器、BN/Dropout、指标零分母、受控实验及答案。压缩：大段源码、接口总表、并发细节，改为固定提交函数索引。

纠正：质心与外接框中心分开；历史 best_acc 与当前权重分开；网页重载发生在 done；“停止/关闭”不作为保存成功保证；关闭探测失败不作为进程退出证明；基本解码校验不称完整安全保证；Origin 不称认证；多标签页共享后端；权重反序列化风险明确；zero-batch 边界具体说明。

版式：数学由数学渲染器绘制、含平方和转置；概率例子总和复算；命令通过变量缩短，代码不拆标识符；统一根目录与 .venv；A4 镜像边距，重复表头、外侧页码、书签与目录；机制图优先矢量，真实灰度图按可读尺寸排入。

确实无法确认及未完成事项

现有 pth 的最初训练环境与完整操作日志未在基线中充分记录，无法从权重和历史元数据恢复；不声称其 99.22% 与最后保存权重完全对应。原历史报告与当前权重的严格对应关系，需当时权重哈希/运行日志才能确认。

固定种子跨环境逐位复现没有验证；多用户部署、未知识别、严格断点恢复、原子保存、最佳权重单独保存、真实亮度质心属于可考虑扩展，不是当前实现。完整真人笔迹泛化、校准与全部浏览器交互未测，不给精度保证。本次 PDF 与实验代码均实际生成，验证范围按证据列出。